Premier University

Department of Computer Science and Engineering



Final year thesis report on

Bangla Handwritten Upazila–District Pair Name Recognition Using Ensemble Learning

A thesis submitted in partial fulfillment of the requirements

for the degree of Bachelor of Science in Computer Science and Engineering, under the guidance of

Mohammad Hasan

Assistant Professor

Department of Computer Science and Engineering

Premier University

Submitted By:

Mojahidul Islam (2104010202323)

Abidul Hoque Alvee (0222210005101142)

Rahat Imroz Ahmed (0222220005101128)

April, 2026

AUTHOR'S DECLARATION OF ORIGINALITY

We hereby declare that the thesis work entitled **"Bangla Handwritten Upazila–District Pair Name Recognition Using Ensemble Learning Model"** submitted to the Premier University, is a record of an original work done by us under the guidance of **Mohammad Hasan**, Assistant Professor, Department of Computer Science & Engineering, Premier University, Chattogram and this work is submitted for partial fulfillment of the degree of Bachelor Science in Computer Science & Engineering. We can assure that the result of this thesis has not been submitted to any other university.

_____

Mojahidul Islam
ID: 2104010202323

_____

Abidul Hoque Alvee
ID: 0222210005101142

_____

Rahat Imroz Ahmed
ID: 0222220005101128

CERTIFICATION

The thesis entitled **"Bangla Handwritten Upazila–District Pair Name Recognition** Using Ensemble Learning Model" submitted by, **Mojahidul Islam**, ID: 2104010202323, **Abidul Hoque Alvee**, ID: 0222210005101142, **Rahat Imroz Ahmed**, ID: 0222220005101128, has been accepted as satisfactory in partial fulfillment of the degree of Bachelor of Science in Computer Science & Engineering (CSE) to be awarded by Premier University, Chattogram.

_____

Professor S. M. Nasrul Quadir
Chairman
Department of Computer Science & Engineering
Premier University, Chattogram

_____

(Thesis Supervisor)
Mohammad Hasan
Assistant Professor
Department of Computer Science & Engineering
Premier University, Chattogram

ACKNOWLEDGMENTS

First and foremost, we thank the Almighty for His boundless grace, which kept us mentally and physically able to complete this challenging work. This thesis would not have taken its final form without the continuous support of Premier University, Chittagong, which gave us the opportunity to pursue our Bachelor's program. We are grateful to the technical and administrative staff of the CSE department for their assistance. The successful completion of this research is not solely our own achievement; it is the result of the cooperation and encouragement of many people.

We extend our deepest gratitude to our supervisor, **Mohammad Hasan**. His profound knowledge of Deep Learning and Machine Learning, along with his eagerness to share it, his patience, humility, understanding, and wise guidance, have been truly remarkable. His advice on preparing manuscripts, reports, and presentations has made us more dynamic and well-organized. We consider him the finest academic mentor we have encountered. We also thank our friends and family for their unwavering support and valuable insights.

We are indebted to everyone who helped us collect the handwritten text for our dataset. Finally, we express our sincere thanks to all our teachers throughout the Bachelor's program. Without their teaching, we could never have acquired the knowledge that forms the foundation of this work.

ABSTRACT

Handwritten address recognition in low-resource languages such as Bangla remains a challenging and practically important problem. Automated recognition of handwritten Bangla upazila–district name pairs can significantly accelerate postal sorting, administrative data digitization, and document processing in Bangladesh. This thesis presents an end-to-end system for Bangla Handwritten Upazila–District Pair Name Recognition (BNHW-UDPNR). A novel dataset, BNHW-UDPNR, was constructed from 320 handwritten pages contributed by 496 unique upazila–district pairs, yielding 9,808 segmented images. An unsupervised text segmentation pipeline—combining Maximally Stable Extremal Region (MSER) detection, Canny edge detection, morphological operations, and connected component analysis—was developed to automatically crop individual name regions. An ensemble model combining Vision Transformer (ViT-Base) and ConvNeXt-Tiny via weighted soft voting is proposed. The proposed ensemble achieves a test accuracy of 94.65%, outperforming baseline models and alternative ensemble strategies. The weighted soft-voting ensemble not only surpasses baseline models but also demonstrates robust generalization, achieving a macro-averaged F1-score of 0.94 and a near-perfect ROC-AUC of 99.99%.

Keywords—Bangla handwritten recognition; upazila–district pair; Vision Transformer; ConvNeXt; ensemble learning; text segmentation; deep learning; transfer learning.

TABLE OF CONTENTS

Declaration **2**

Author's Declaration of Originality **3**

Certification **4**

Acknowledgments **5**

Abstract **6**

Chapter 1: Introduction **7**

1.1 Introduction 7
1.2 Related Works 7
1.3 Problem Statement 9
1.4 Motivation 9
1.5 Research Objectives 9
1.6 Applications 10
1.7 Thesis Organization 10

Chapter 2: Background Study **11**

2.1 Definition of Data 11
2.2 Definition of Dataset 11
2.3 Annotation of Data 12
2.4 What is OCR 12
2.5 Applications of OCR 12
2.6 Challenges in Bengali OCR 12
2.7 Applications of Handwritten Text Recognition In Bangladesh 13
2.8 Computer Vision 14
2.9 Deep Learning Architectures for Image Classification 14
2.10 Overview of Vision Transformers 17
2.11 Transfer Learning 23
2.12 Overview of Ensemble Learning 24
2.13 Dataset Construction in Low-Resource Settings 27
2.14 Research Gap and Motivation 27

Chapter 3: Research Methodology **29**

3.1 Overall Working Process 29
3.2 Data Collection 30
3.3 Text Segmentation 31
3.4 Data Preprocessing 28
3.5 Classification 32
3.6 Environment Setup 33

Chapter 4: Text Segmentation **34**

4.1 BNHW-UDPNR: Data Annotation Process 34
4.2 Automatic Text Segmentation 37

4.3 Observation of Datasets 44
4.4 Structure of our dataset 46
Chapter 5: Data Preprocessing 47
5.1 Image Preprocessing 47
5.2 Data Augmentation 47
Chapter 6: Proposed Model Architecture 49
6.1 Working Procedure of the Proposed Model 49
6.2 Two-Stage Training Strategy 50
6.3 Vision Transformer (ViT-Base) Architecture 50
6.4 ConvNeXt-Tiny-Based Classification Architecture 52
6.5 Custom Classification Head 55
6.6 Weighted Soft Voting 55
6.7 Model Architecture 56
6.8 Conclusion 57
Chapter 7: Experiments and Results 58
7.1 Experimental Setup 58
7.2 Hyperparameter Configuration 59
7.3 Evaluation Metrics 65
7.4 Model Performance Comparison 62
7.5 Model Test Accuracy Comparison 65
7.6 Training and Validation Curves 65
7.7 ROC-AUC Analysis 66
7.8 Effects of Activation Function 67
7.9 Correct and Misclassified Examples 68
7.10 Scope and Contributions 69
7.11 Conclusion 70
Chapter 8: Conclusion and Future Work 71
8.1 Conclusion 71
8.2 Discussion 71
8.3 Future Work 72

References 73

CHAPTER 1: INTRODUCTION

1.1 Introduction

The digitization of handwritten documents has long been a challenge in pattern recognition and computer vision. In densely populated countries like Bangladesh, many administrative processes—such as postal routing, land registry, school enrollment, and census data entry—still rely on handwritten forms. One of the most important identifiers on these forms is the upazila–district name pairs: Bangladesh is divided into 8 divisions, 64 districts, and 496 upazilas, and accurately recognizing these place names is essential for automating document routing and indexing.

Bangla (Bengali) is the seventh most spoken language worldwide, with over 230 million native speakers [1]. Despite its widespread use, research on Bangla optical character recognition (OCR) and handwriting recognition hasn't advanced as quickly as for Latin-script languages. The Bangla script offers unique challenges because it is an abugida with a complex array of base characters, vowel diacritics (matras), consonant conjuncts, and a special matra line (shirorekha) that connects characters at the top [13]. Handwritten Bangla adds even more complexity with individual stylistic variations, different stroke thicknesses, skew, and noise [14, 15].

Existing Bangla handwriting recognition systems typically focus on isolated characters or basic word recognition [2, 3]. Recognizing full multi-word address-level strings—such as upazila–district pairs—remains less explored. The recent rise of large-scale pre-trained deep learning models, especially Vision Transformers (ViT) [4] and modern convolutional networks like ConvNeXt [5], provides promising opportunities for high-accuracy classification even with moderate amounts of task-specific data through transfer learning from extensive natural-image datasets like ImageNet [18].

In this study, we proposed an ensemble approach combining a Vision Transformer (ViT-Base) and a modern convolutional network (ConvNeXt-Tiny). The methodology includes creating a labeled dataset (BNHW-UDPNR) of upazila–district pairs, an automatic text segmentation pipeline, and a two-stage fine-tuning strategy. The two models share a custom MLP head and are combined via weighted soft voting, achieving robust recognition of handwritten Bangla administrative names.

1.2 Related Works

Many recent studies have concentrated on creating systems to recognize Bangla characters and text in handwritten scripts. This section reviews key contributions to Bangla handwritten recognition, organized by the methodological approach.

1.2.1 Deep learning approaches

Deep learning techniques have significantly advanced the performance of Bangla handwritten recognition systems. Several CNN-based architectures have been proposed for character recognition. S. M. Mahbubur Rahman et al. [6] introduced a deep convolutional neural network (DCNN) that achieved 93.07% accuracy on their custom dataset, outperforming conventional CNN and VGG-16 models [19]. Similarly, M. A. Hamid et al. [7] developed a modified ResNet-34 architecture [20] that attained 97% overall accuracy and 99.02% for numeric characters, demonstrating strong generalization across vowels, consonants, and digits.

Beyond basic characters, Md. Rashedul Islam et al. [10] proposed RATNet, a hybrid deep learning model specifically designed to recognize characters, modifiers, and compound forms. By incorporating statistical analysis of character frequency, their model achieved accuracies ranging from 97.70% to 99.66%, outperforming VGG-16 [19] and ResNet-50 [20].

At the word level, Md. H. Zisad et al. [8] introduced the BNHW-UDPNR dataset along with a CNN-based classifier for handwritten Bengali district names. Using 7,040 samples, their system addressed the curved nature of Bangla script and achieved 98.44% test accuracy. In a different approach, A. Mollah et al. [11] explored feature-based recognition of handwritten Bangla city names, focusing on extracting discriminative features for improved classification performance.

For end-to-end OCR and scene text extraction, two notable pipelines have been developed. T. Tazin et al. [3] combined YOLO for detection and EfficientNet-B4 for recognition, achieving 93.87% accuracy for grapheme roots and over 98% for diacritics, with a Word2Vec-based correction mechanism significantly reducing the Character Error Rate (CER). Similarly, M. Kowsher et al. [9] presented an end-to-end framework for Bangla signboard detection and classification, integrating YOLOv9-s for text detection, Tesseract OCR for text extraction, and a fine-tuned BERT model [45] for classification, achieving 99.1% detection accuracy on a dataset of over 16,000 images.

1.2.2 Machine learning approaches

Md. Tarek Habib et al. [12] provided a comparative overview of various classification algorithms for Bangla Handwritten Character Recognition (BHCR) to help researchers select suitable methods. They created a new dataset of 2,500 vowel samples from 150 volunteers and applied histogram adjustment as a preprocessing step. After evaluating seven common algorithms, they found that the Artificial Neural Network (ANN) performed best overall, while Logistic Regression (LR) exhibited high sensitivity and specificity.

Additionally, A. Mollah et al. [11] explored the use of the Histogram of Oriented Gradients (HOG) descriptor for feature extraction in recognizing handwritten Bangla city names. They used a dataset of 18,000 samples across 120 words categories and trained a Support Vector Machine (SVM) classification model. Their experimental results showed that the HOG-based approach effectively distinguishes word categories, reaching an accuracy of 86.80%.

1.2.3 OCR Systems and Text Extraction

The work by T. Tazin et al. [3] introduced a scalable and effective OCR pipeline for Bangla handwritten documents to address the complexities of modifiers and compound characters. They used the YOLO model for character detection and EfficientNet-B4 for recognition, reaching accuracies of 93.87% for grapheme roots and over 98% for diacritics. By adding a Word2Vec-based spelling correction layer, they successfully lowered the Character Error Rate (CER) from 10.37% to 2.47%.

Md. Kowsher et al. [9] developed an automated Bangla signboard detection and classification system for urban information access. The system focuses on extracting text from natural scenes and categorizing businesses to support smart city development. Their four-step pipeline uses YOLOv9-s to locate text regions, Tesseract OCR to extract text, and a fine-tuned BERT model [45] for classification. Trained on a large dataset of 16,402 images, it achieved high accuracy in all stages, including 99.1% success in signboard detection.

1.3 Problem Statement

Currently, there is no publicly available dataset specifically for recognizing handwritten Bangla upazila-district pairs. Most existing research on Bangla handwritten recognition primarily focuses on isolated characters or digits [2, 6, 7, 10, 12], with limited attention to word-level recognition. General OCR systems find it difficult to recognize long handwritten words due to the complex, curvy nature of the Bangla script, the presence of compound characters [17], and the way characters often join inside words—features that are especially challenging in handwritten text [14, 15]. Moreover, many upazila names across different districts have identical or very similar spellings, making it harder to identify the correct administrative unit without also recognizing the district name.

To address these issues, this research finds the gap to create a novel dataset consisting of handwritten strings of complete upazila-district pairs, capturing the natural variability of real-world handwriting from different writers, and develops a state-of-the-art deep learning model capable of accurately recognizing both the upazila and its corresponding district from a single handwritten sample with high accuracy.

1.4 Motivation

The potential for handwritten recognition applications across different languages is substantial. Bangla, the primary language of Bangladesh, is used in many areas, including government offices, postal services, the national ID system, vehicle license plates, and public signs. Most physical documents with Bangla script encountered in daily tasks are now handled digitally, creating a significant opportunity to implement handwritten recognition systems.

However, several challenges hinder progress in this area. First, there remains a significant lack of a well-structured dataset for Bangla handwritten text. Second, the nature of the Bangla language itself introduces complications due to its curvilinear letters and compound characters [13, 17]. These challenges are worsened by the natural variability of individual handwriting, leading to words and letters appearing in diverse forms across various writers [14]. Because of these factors, we cannot fully leverage the benefits of automatic handwritten recognition systems, which could be useful in many aspects of daily life in Bangladesh.

To address this gap, we introduce a new dataset and recognition technique for handwritten names of administrative units in Bangladesh. Our dataset includes paired handwritten samples featuring an upazila and its associated district. The inspiration for creating this system arises from practical needs in Postal Automation and the Digitization of Government Records. In real handwritten documents—such as mail, land records, and official forms—the names of upazila and district frequently appear side by side. Developing a system that can automatically recognize such paired administrative names would significantly streamline document-processing workflows and reduce manual data-entry effort.

1.5 Research Objectives

Primary Objective:

The primary objective of this research is to develop a system that can accurately identify handwritten Bangla upazila names along with their corresponding district names

Secondary Objectives:

1. To create a novel dataset (BNHW-UDPNR) comprising handwritten samples for upazila-district pairs in Bangladesh, capturing a variety of handwriting styles across different age groups and professions.
2. To develop and execute an unsupervised text segmentation pipeline that accurately extracts individual upazila–district pair regions from multi-column handwritten pages.
3. To develop a deep learning ensemble model for recognizing handwritten Bangla upazila–district pairs.
4. To evaluate and compare the performance of our proposed model with other baseline deep learning models.

1.6 Applications:

- Postal & Courier Automation
- Administrative Document Digitization
- Logistics & Supply Chain
- E-commerce Address Verification
- Bank Check Processing

1. 7 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 provides a comprehensive review of Background study, covering Bangla handwritten text recognition, text segmentation techniques, deep learning architectures for image classification, and ensemble learning methods.

Chapter 3 describes the research methodology, including the overall system architecture, data collection process, and experimental framework.

Chapter 4 presents the automatic text segmentation pipeline, covering pre-processing steps, segmentation techniques (column detection, row grouping, connected component analysis, bounding box merging, and affine cropping), and segmentation observations.

Chapter 5 details the data preprocessing pipeline, including dataset annotation, resizing to 224×224 pixels, ImageNet normalization, train-validation-test splitting, and balancing augmentation.

Chapter 6 presents the proposed model architecture, including the custom classification head, ViT-Base, and ConvNeXt-Tiny architectures, ensemble methods, training strategy, and regularization techniques.

Chapter 7 reports experimental results, including model parameter comparison, training curves, individual model performance, ensemble comparisons, detailed classification analysis, and ablation studies.

Chapter 8 concludes the thesis with a summary of findings, a discussion of limitations, and suggestions for future research.

CHAPTER 2: BACKGROUND STUDY

2.1 Definition of Data

Data consists of raw facts, observations, or measurements collected from various sources that can be processed to generate meaningful information. In computer science and machine learning, data serves as the basic input for computational models and algorithms. Data can take different forms such as numbers, text, images, audio, or video, and it represents real-world phenomena that can be analyzed and understood.

In machine learning systems, data is used to train models so they can learn patterns, relationships, and structures within the information. By analyzing large amounts of data, machine learning algorithms can make predictions, classifications, or decisions without explicit programming.

2.1.1 Types of Data

Data can usually be divided into two main types: qualitative and quantitative.

2.1.1.1 Qualitative Data

Qualitative data refers to non-numeric information that describes characteristics or qualities. This type of data is usually descriptive and cannot be directly measured with numerical values. Examples include colors, labels, categories, or textual descriptions. Specific examples of qualitative data are city names, vehicle types, gender categories, and text-based information. Qualitative data is frequently used in classification tasks where objects are sorted into different classes [2].

2.1.1.2 Quantitative Data

Quantitative data refers to numerical information that can be measured and expressed using numbers. This type of data can be analyzed using statistical and mathematical techniques. Quantitative data can be further divided into discrete data (countable numerical values, e.g., number of students) and continuous data (measurable values within a range, e.g., height, temperature). Quantitative data is widely used in statistical analysis, machine learning, and scientific research [12].

2.2 Definition of Dataset

A dataset is a structured collection of related data organized for analysis and processing. In machine learning and data science, datasets are used to train, validate, and evaluate models. Datasets usually consist of multiple data points, each containing several attributes or features describing a particular observation. These features assist machine learning algorithms in identifying patterns and relationships within the data. Datasets can be stored in various formats such as spreadsheets, databases, images, or text collections [8].

2.2.1 Purpose of the Dataset

Datasets serve several important purposes in machine learning and data analysis. They are used to train machine learning algorithms to identify patterns, test the performance of models on separate evaluation sets, fine-tune model parameters through validation, and allow researchers to examine relationships between variables. A well-organized dataset is essential for developing reliable machine learning models [3].

2.3 Annotation of Data

Data annotation is a critical process in machine learning where raw data is labelled or tagged so that models can learn from it. Annotated data provides the ground truth that enables machine learning algorithms to recognize patterns and make predictions. In supervised learning, annotated datasets are essential because the model requires labeled examples to learn the mapping between inputs and outputs [12].

2.3.1 Data Annotation

Data annotation involves attaching meaningful labels or metadata to raw data so machine learning models can understand and learn from it. For instance, in image recognition, images may be tagged with object categories, while in text processing, sentences may be marked with sentiment labels. Precise annotation is essential for good model performance because incorrect labels can result in poor predictions [8].

2.4 What is OCR

Optical Character Recognition (OCR) is a technology that converts images containing text into machine-readable digital text. OCR systems analyze scanned documents, photographs, or handwritten text and recognize the characters present in the image. OCR works by identifying patterns that correspond to letters, numbers, and symbols in an image. This technology allows computers to extract textual information from printed or handwritten documents [27]. OCR is commonly used in applications like document digitization, automatic form processing, and handwriting recognition.

2.5 Applications of OCR

OCR technology has many real-world uses across different industries. Key uses include digitizing documents, automating data entry, processing bank checks, recognizing license plates, sorting postal mail, and preserving historical documents. OCR systems enable quick conversion of printed documents to digital formats, reducing manual data entry.

2.6 Challenges in Bengali OCR

Despite major advances in OCR technology, recognizing Bengali text remains difficult due to the complexity of the Bengali script. The main challenges are outlined below.

- **Large number of characters and compound characters:** The Bengali script contains a large set of basic characters, including vowels and consonants, as well as numerous compound characters formed by combining multiple consonants [17]. These compound characters create complex glyph structures that significantly increase the number of patterns an OCR system must recognize. As a result, the recognition process becomes more challenging compared to languages with smaller character sets [13].
- **Complex character shapes:** Many Bengali characters have intricate shapes with curves, loops, and strokes that can appear visually similar. These complex structures make it difficult for OCR systems to distinguish characters, especially when image quality is low or when characters are written quickly [14].
- **Variability in handwriting styles:** Handwritten Bengali text differs greatly among individuals because of writing habits, stroke thickness, and writing speed. This variation

makes it hard for OCR models to learn consistent character patterns, so they need to be trained on large and varied datasets [15].

- **Presence of modifiers and diacritics:** Bengali characters frequently include modifiers and diacritical marks that appear above, below, or beside the base character. Their varying positions and shapes make it challenging for OCR systems to accurately detect and associate them with the correct base characters [16].
- **Similar character structures:** Many Bengali characters have very similar visual structures that differ only by small strokes or minor variations in shape, easily confusing OCR systems when noise, blur, or distortions are present. Accurate recognition, therefore, requires highly discriminative feature extraction techniques [17].

These challenges make Bengali OCR systems more difficult to develop than those for languages with simpler scripts.

2.7 Applications of Handwritten Text Recognition in Bangladesh

The ability to automatically recognize handwritten Bangla upazila–district pairs has direct and valuable applications across several real-world domains in Bangladesh.

- **Postal Automation:** The Bangladesh Postal Department processes millions of handwritten letters and parcels each year. Automatically extracting upazila and district names from envelope addresses would enable automated sorting, reduce delivery errors, and significantly speed up mail processing.
- **Land Registry and Legal Documents:** Land registration offices still rely on handwritten forms and deeds. Digitizing these documents with automated recognition of administrative names would facilitate faster record retrieval, reduce disputes, and support digital land management systems.
- **Census and National ID Systems:** During national censuses, enumerators collect handwritten data that includes district and upazila information. Automated recognition would accelerate data entry and improve the accuracy of population statistics.
- **Banking and Cheque Processing:** Banks in Bangladesh process a large volume of handwritten cheques. Automatically recognizing payee names and locations could streamline cheque clearing and reduce manual verification efforts.
- **E-commerce and Logistics:** With the rapid expansion of online shopping, handwritten address labels on parcels need to be read quickly and accurately. Recognizing upazila–district pairs would help courier services route packages correctly and on time.
- **Educational Administration:** School admission forms, exam registration sheets, and other administrative documents often contain handwritten district and upazila names. Automating data entry from these forms would save time and reduce human error.

Despite these pressing needs, no existing system reliably recognizes handwritten Bangla upazila–district pairs at scale. This thesis directly addresses this gap by developing a robust ensemble model and a dedicated dataset, thereby enabling practical deployment in these application areas.

2.8 Computer Vision

Computer Vision is a field of artificial intelligence that enables machines to interpret and understand visual information from images and videos. The aim of computer vision is to enable computers to extract meaningful information from visual inputs and make decisions based on that information. Computer vision systems usually involve image acquisition, image preprocessing, feature extraction, object recognition, and classification. Modern computer vision applications often rely on deep learning models, such as Convolutional Neural Networks (CNNs) and Vision Transformers (ViT), to achieve high accuracy in tasks such as image classification, object detection, and scene understanding [26]. Computer vision technologies are widely applied in autonomous vehicles, medical imaging, facial recognition, and surveillance systems.

2.9 Deep Learning Architectures for Image Classification

2.9.1 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) have been the dominant paradigm for image classification since the landmark AlexNet [18] demonstrated that deep CNNs trained on GPUs could substantially outperform handcrafted feature methods in the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC). The key design principles of CNNs—local receptive fields, shared weights, and hierarchical feature learning—provide strong inductive biases for visual data, encoding the assumptions of translation equivariance and local spatial correlation.

Subsequent architectures progressively deepened and widened CNNs: VGGNet [19] demonstrated the value of depth with small (3×3) filters; ResNet [20] introduced residual connections, enabling the training of networks exceeding 100 layers; and DenseNet [21] connected every layer to all subsequent layers to maximize feature reuse. Each advance improved accuracy on standard benchmarks while also improving representational capacity for downstream tasks via transfer learning.

For handwritten text, CNNs naturally capture stroke-level features in the early layers and character-level structures in the deeper layers, making them well-suited for character and word recognition tasks. However, standard CNNs are inherently local—their receptive fields grow only slowly with depth—and may miss long-range contextual relationships that are important for distinguishing visually similar words.

2.9.1.1 Types of Convolutional Neural Networks

Over time, many CNN architectures have been developed to improve performance and efficiency.

2.9.1.1.1 AlexNet

AlexNet, proposed by Krizhevsky et al. in 2012, marked a major breakthrough in deep learning by winning the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 with a significant margin over traditional methods [18]. Key features include a deep architecture with eight layers, ReLU activation functions, dropout layers to reduce overfitting, and GPU-accelerated training. AlexNet demonstrated the effectiveness of deep CNNs in large-scale image classification and played a crucial role in popularizing deep learning.

2.9.1.1.2 VGGNet

VGGNet, developed by the Visual Geometry Group at the University of Oxford in 2014, introduced deeper network architectures using very small 3×3 convolution filters [19]. Instead of utilizing large convolution kernels, VGGNet stacks multiple small convolution layers (16 or 19 layers) to increase network depth, enhancing the model's ability to learn complex features while keeping computational complexity manageable.

2.9.1.1.3 ResNet

ResNet (Residual Network), introduced by He et al. in 2015, addresses the challenge of training very deep neural networks [20]. As networks grow deeper, vanishing gradients make training difficult. ResNet solves this problem using residual (skip) connections. A residual block enables the input to bypass one or more layers and be added directly to the output, as defined in (18). ResNet architectures can have 50, 101, or more layers.

$$y = F(x) + x \tag{1}$$

where $F(x)$ represents the learned residual mapping.

2.9.1.1.4 DenseNet

DenseNet (Dense Convolutional Network), introduced by Huang et al. in 2017, enhances information flow between layers [21]. Unlike traditional CNNs, where each layer only receives input from the previous layer, DenseNet connects each layer to all subsequent layers in a feed-forward way. Each layer receives feature maps from all preceding layers. This dense connectivity improves gradient flow, promotes feature reuse, and reduces the number of parameters compared to traditional CNNs.

2.9.1.1.5 **ConvNeXt: Modern CNN Design**

Liu et al. [5] proposed ConvNeXt as a modernized CNN designed to match or exceed the performance of vision transformers by systematically incorporating transformer-inspired design choices into a pure convolutional framework. Starting with a standard ResNet-50, the authors implemented a sequence of improvements: employing larger (7×7) depthwise convolutions (similar to ViT's attention span), stage-wise channel scaling ratios, using layer normalization instead of batch normalization, adopting GELU activation [22], reducing the number of activation functions and normalization layers, and utilizing inverted bottleneck MLP blocks.

The resulting ConvNeXt-Tiny achieves ImageNet accuracy comparable to Swin-Tiny while retaining the simplicity of a standard CNN. Key advantages for the present task are its strong local feature extraction capabilities—important for recognizing Bangla stroke patterns—and its relatively compact parameter count (28M vs. 86M for ViT-Base), making it computationally efficient without sacrificing accuracy.

ConvNeXt has been successfully applied to document understanding and OCR-related tasks, confirming its suitability for text image classification. The combination of ViT-Base and ConvNeXt-Tiny as ensemble components exploits their complementary inductive biases: ViT captures global relational context while ConvNeXt excels at local spatial pattern recognition.

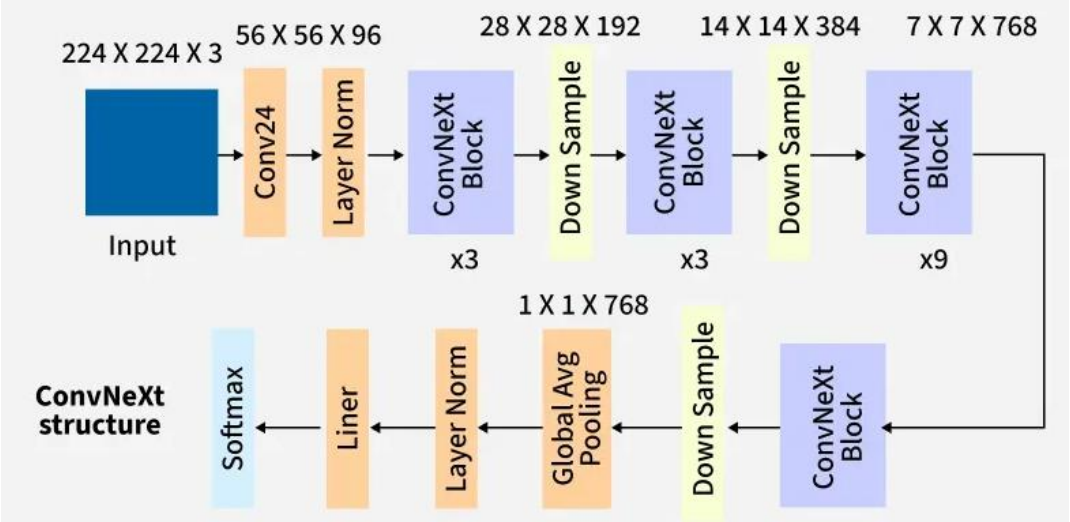


Figure 2.1: ConvNext model Architecture [46]

2.9.2.1 Key Architectural Innovations

Macro Design:

- **Stage ratio adjustment**: The number of blocks in each stage was modified from the ResNet pattern (3,4,6,3) to match transformer-like ratios (3,3,9,3)
- **Stem cell**: The original 7×7 convolution with stride 2 was replaced with a 4×4 convolution with stride 4, similar to the patch embedding layer in ViT
- **Downsampling**: Separate downsampling layers using 2×2 convolutions were introduced between stages

Micro Design:

- **Depthwise convolutions**: Large kernel (7×7) depthwise convolutions were adopted, analogous to the attention span in transformers
- **Inverted bottleneck**: Following transformer MLP design, an inverted bottleneck structure (expand → project) was implemented, expanding channels by a factor of 4 before projecting back
- **Normalization**: Batch normalization was replaced with layer normalization throughout the network, mirroring transformer practice
- **Activation**: ReLU was replaced with GELU activation for smoother non-linearity
- **Downsampling**: Fewer activation functions and normalization layers were used, creating cleaner information paths

Training Techniques:

- **Stochastic depth**: Randomly dropping entire blocks during training with probability proportional to block depth
- **AdamW optimizer**: Replacing standard SGD with the AdamW optimizer

- **Data augmentation**: Employing modern augmentation strategies including Mixup and CutMix

2.9.2.2 ConvNeXt-Tiny Architecture

ConvNeXt-Tiny, the variant used in this thesis, consists of four stages with increasing channel dimensions: The resulting architecture achieves ImageNet accuracy comparable to Swin-Tiny (a transformer-based model) while retaining the simplicity of a standard CNN. Key advantages for handwritten text recognition include:

1. **Strong local feature extraction**: Depthwise convolutions effectively capture stroke patterns and character shapes
2. **Efficient parameter usage**: ConvNeXt-Tiny has only 28M parameters compared to 86M for ViT-Base
3. **Hierarchical feature learning**: Multi-stage design captures features at multiple scales
4. **Proven effectiveness**: ConvNeXt has been successfully applied to document understanding and OCR-related tasks, confirming its suitability for text image classification

2. 10 Overview of Vision Transformers

Although deep learning models like CNNs have dominated computer vision, they also face limitations in image recognition. CNNs primarily focus on localized patterns such as edges, textures, and shapes, creating a strong bias toward local features. Fixed-size kernels cause CNNs to treat features uniformly across the entire image, making it difficult to scale with increasing image sizes. Vision Transformers, on the other hand, effectively capture global context and long-range dependencies. They treat images as sequences of patches and use an attention mechanism to model relationships between them—allowing them to view the whole image at once and identify which parts are most important [4].

2.10.1 Attention Mechanism

The attention mechanism is a fundamental concept in modern deep learning that allows a model to focus on the most relevant parts of the input while processing information. Instead of treating all parts of the input equally, attention assigns different importance weights to different elements, enabling the model to prioritize the most informative features.



Figure 2.2: Attention Mechanism Example

The figure illustrates how the attention mechanism focuses on the most relevant parts of a sentence when interpreting contextual meaning. In the sentence *"She is eating a green apple,"*

The attention mechanism became widely known after the influential paper "Attention Is All You Need" by Vaswani et al. [22], which introduced the Transformer architecture entirely

based on attention without recurrent or convolutional operations. The attention mechanism operates using three key components: Query (*Q*), Key (*K*), and Value (*V*)

- **Query (Q):** represents the element that is searching for relevant information.
- **Key (K):** represents the elements that can provide relevant information.
- **Value (V):** represents the actual information associated with each key.

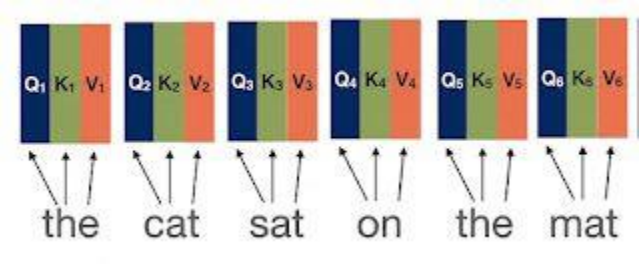


Figure 2.3: The attention mechanism represents each word in a sentence

The figure illustrates how the attention mechanism represents each word in a sentence using three vectors: Query (Q), Key (K), and Value (V). In the example sentence *"the cat sat on the mat,"* every word is transformed into its corresponding query, key, and value vectors.

The similarity between the Query and Key vectors determines how much attention should be given to each Value. This relationship is computed using the scaled dot-product attention formula given in (1), where *Q*, *K*, and *V* are the query, key, and value matrices, and d^k is the dimensionality of the key vectors used for scaling.

$$Attention(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

2.10.2 Transformer Architecture

The Transformer architecture was introduced by Vaswani et al. **[22]** in the seminal 2017 paper "Attention Is All You Need." Unlike traditional sequence modelling architectures such as RNNs or CNNs, the Transformer relies entirely on attention mechanisms, enabling efficient capture of long-range dependencies and highly parallel computation during training. The architecture consists of two main components: an Encoder and a Decoder.

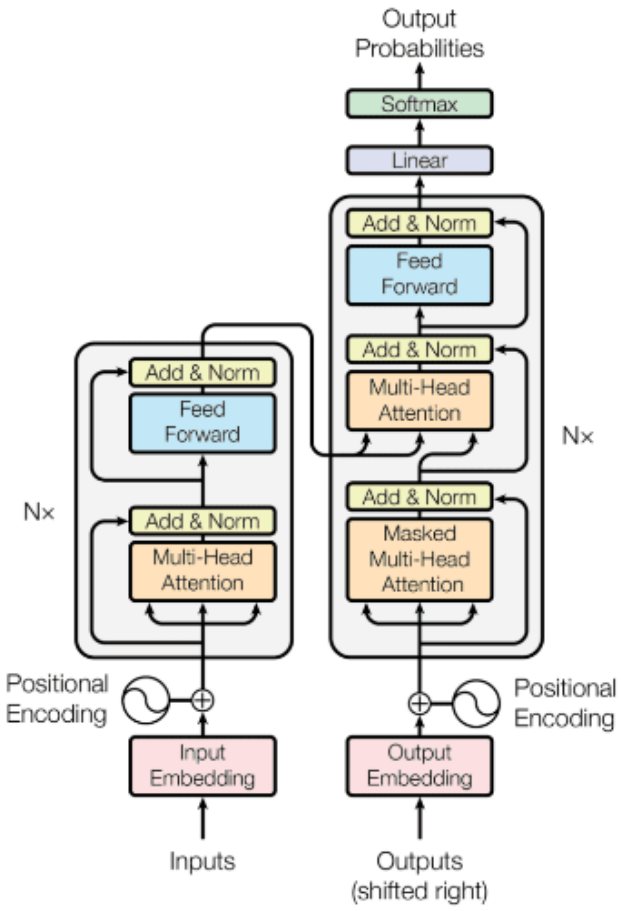


Figure 2.4: Transformer Model Architecture [22]

2.10.2.1 Transformer Encoder

The encoder processes the input sequence and generates feature representations that capture contextual relationships between input tokens. Before entering the encoder, each input token is converted into a numerical vector via input embedding, and positional encoding is added to preserve token order information. Each encoder layer consists of: (1) Multi-Head Self-Attention, where each token attends to every other token allowing the model to capture relationships regardless of distance; (2) Add & Normalize, a residual connection followed by layer normalization to stabilize training; (3) Feed-Forward Neural Network, a position-wise network with two fully connected layers and a nonlinear activation function; and (4) another Add & Normalize step. Multiple stacked encoder blocks form the complete encoder.

2.10.2.2 Transformer Decoder

The decoder generates the output sequence based on the encoded representations. Similar to the encoder, the decoder receives output embeddings combined with positional encodings. Each decoder layer contains three components: (1) Masked Multi-Head Self-Attention, which prevents the decoder from accessing future tokens during training; (2) Encoder-Decoder Attention, where the decoder attends to encoder output representations to incorporate input context; and (3) a Feed-

Forward Neural Network. Residual connections and layer normalization are applied after each sub-layer.

2.10.2.3 Full Process

Input embeddings are combined with positional encoding to incorporate word order information and processed through multiple encoder layers. On the decoder side, the output sequence is shifted right before embedding to ensure each token is predicted based only on previously generated tokens. The decoder applies masked self-attention, encoder-decoder attention, and feed-forward networks sequentially. Finally, a linear transformation followed by a softmax function produces probability distributions over possible output tokens [22].

2.10.3 Vision Transformer (ViT)

The Vision Transformer (ViT) was introduced by Dosovitskiy et al. **[4]** in the paper "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale" (2020). The key idea is to treat an image as a sequence of smaller patches, analogous to how words are treated as tokens in NLP. Instead of using convolutional operations, ViT divides an image into fixed-size patches and processes them using the transformer encoder architecture.

For a 224×224 pixel input image with a patch size of 16×16, the image is divided into $(224/16)^2 = 196$ patches. Each patch is flattened into a vector, projected into a higher-dimensional embedding space, and then processed by a transformer encoder that learns relationships between patches using the attention mechanism. The final output representation is passed through a classification head to produce the prediction.

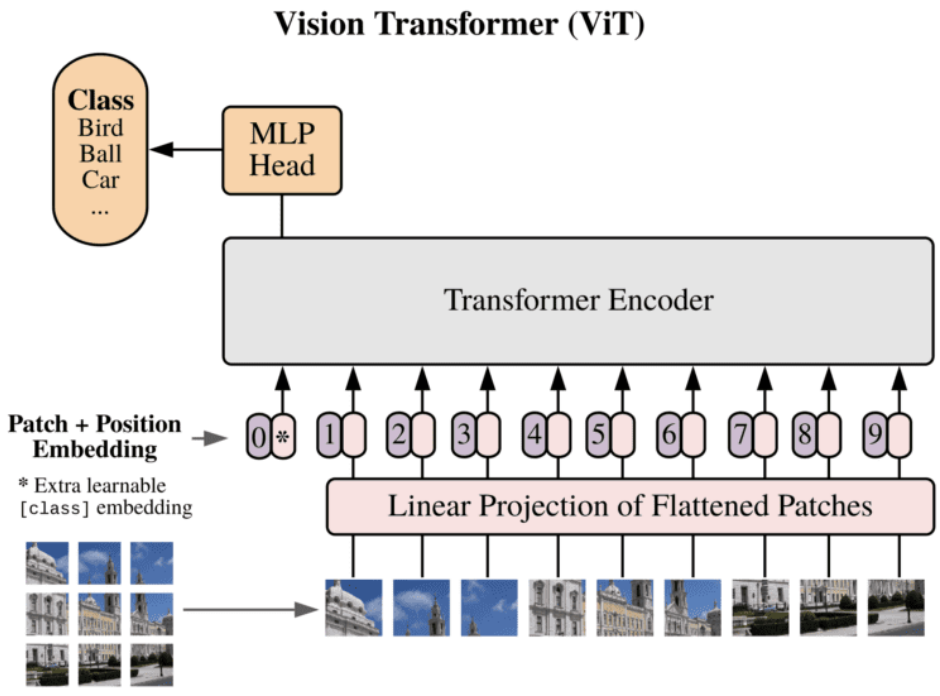


Figure 2.5: Vision Transformers: dividing image into patches [4]

2.10.3.1 Image Patch Extraction

The first step in ViT is to divide the input image into smaller, fixed-size patches. Rather than processing the entire image at once, the model processes these patches as individual tokens, treating the image as a sequence of visual tokens analogous to words in a sentence [4].

2.10.3.2 Linear Projection of Flattened Patches (Patch Embedding)

After extracting patches, each patch is flattened into a one-dimensional vector and then projected into a higher-dimensional embedding space using a linear layer. This patch embedding process converts raw pixel values into numerical feature representations that can be processed by the transformer.

2.10.3.3 Positional Embedding

Since transformers do not inherently capture spatial relationships between patches, positional embeddings are added to patch embeddings to preserve spatial information. These embeddings encode the relative position of each patch within the image. A special classification token (CLS token) is also appended to the sequence; it aggregates information from all patches and is later used for classification [4].

2.10.3.4 Transformer Encoder

The sequence of embedded patches is passed through multiple transformer encoder layers, each consisting of Layer Normalization, Multi-Head Self-Attention, a Feed-Forward Neural Network (MLP), and Residual Connections. The self-attention mechanism enables the model to capture relationships between different patches, allowing the network to understand global context across the entire image.

2.10.3.5 MLP Classification Head

After processing through the transformer encoder layers, the output representation corresponding to the CLS token is extracted and passed through a Multi-Layer Perceptron (MLP) head, which produces the final classification probabilities [4].

2.10.4 Working Mechanism of Vision Transformer

The Vision Transformer processes an input image by dividing it into multiple small patches. Each patch is flattened and passed through a linear projection layer to produce a vector representation. Patch embeddings are combined with positional embeddings to preserve spatial information. The sequence of embedded patches is then fed into the Transformer Encoder consisting of repeated layers of layer normalization, multi-head self-attention, and a feed-forward MLP. Through self-attention, the model learns relationships between different image patches and captures global contextual information. After processing through the encoder layers, the learned representation is passed to an MLP head, which produces the final classification output.

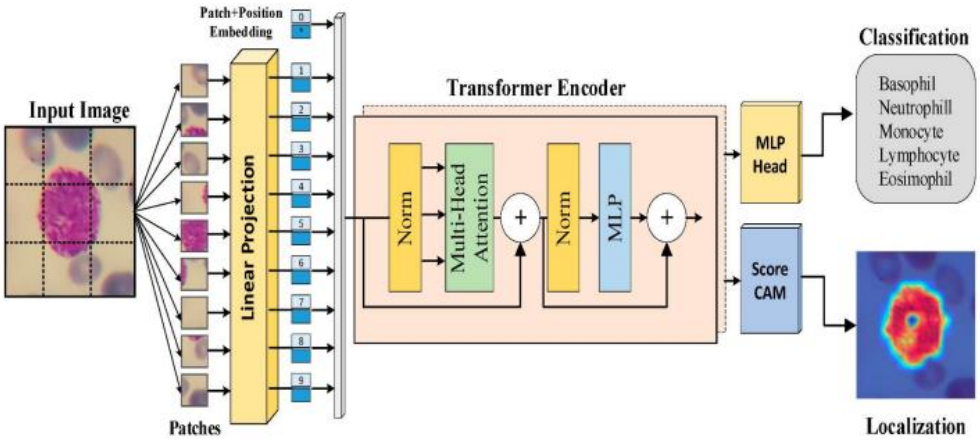


Figure 2.6: Working Mechanism of Vision Transformer [47]

2.10.5 Types of Vision Transformers

Several variations of ViT architectures have been proposed to improve performance, scalability, and efficiency. Among these, ViT-Base is one widely used model.

2.10.5.1 ViT-Base

ViT-Base is one of the standard configurations of the Vision Transformer introduced by Dosovitskiy et al. **[4]**. In this architecture, an input image is divided into fixed-size 16×16pixel patches, each treated as a token processed by a transformer encoder. ViT-Base typically consists of 12 transformer encoder layers, 12 attention heads, and an embedding dimension of 768. Because ViT-Base relies entirely on transformer encoders without convolution operations, it can model global dependencies across the entire image. ViT-Base has demonstrated strong performance in image classification tasks when trained on large datasets.

2.10.6 Performance Metrics for Vision Transformers

Evaluating the performance of image classification models such as Vision Transformers requires appropriate quantitative metrics. One of the most commonly used metrics is classification accuracy, which measures the proportion of correctly predicted samples [42]. In transformer-based vision models evaluated on large-scale datasets such as ImageNet, two widely used metrics are Top-1 accuracy and Top-5 accuracy [18].

2.10.6.1 Top-1 Accuracy

Top-1 accuracy measures whether the model's most confident prediction matches the true class label. The model predicts a list of class probabilities, and the class with the highest probability is considered the final prediction. Top-1 accuracy is defined in (2). For example, if a model evaluates 100 images and correctly predicts the label of 90 images as the top prediction, the Top-1 accuracy is 90 %. This metric provides a strict evaluation since it considers only the single most confident prediction.

$$Top-1\ Accuracy\ =\ \frac{Number\ of\ Correct\ Predictions}{Total\ Number\ of\ Samples} \quad (3)$$

2.10.6.2 Top-5 Accuracy

Top-5 accuracy measures whether the correct class label appears within the top five predicted classes. This metric is particularly useful in large classification problems where many classes may have similar visual features. Top-5 accuracy is defined in (3). This metric provides a more flexible evaluation compared to Top-1 accuracy, accounting for cases where the correct class is among the most probable predictions [30].

$$Top-5\ Accuracy\ =\ \frac{Number\ of\ Samples\ Where\ True\ Label\ Appears\ in\ Top\ 5}{Total\ Number\ of\ Samples} \quad (4)$$

2. 11 Transfer Learning

Transfer learning is a machine learning technique where a model developed for a source task is reused as the starting point for a target task [24]. This approach is particularly valuable when the target domain has limited labelled data, as it leverages knowledge learned from large-scale datasets to improve performance on data-scarce tasks.

The fundamental principle behind transfer learning is that neural networks learn hierarchical feature representations. Early layers capture general features (edges, textures, color blobs) that are applicable across many visual domains, while later layers learn task-specific features [23]. By transferring weights from a model pretrained on a large dataset, practitioners can:

- **Reduce training time**: Models start from meaningful weights rather than random initialization
- **Improve generalization**: Pretrained features provide regularization that helps prevent overfitting
- **Achieve better performance with limited data**: Domain-specific features can be learned with fewer target samples

2.11.1 ImageNet Pretraining

ImageNet [18] is a large-scale visual database containing over 1.2 million natural images across 1,000 object categories. Since the introduction of AlexNet [18] in 2012, ImageNet has become the de facto standard for pretraining image classification models. The diversity and scale of ImageNet enable models to learn rich, general-purpose visual features that transfer effectively to downstream tasks.

Yosinski et al. [23] conducted a seminal study on the transferability of features in deep neural networks, demonstrating that:

1. Features learned on ImageNet transfer well to other visual tasks, with early layers being nearly universally applicable
2. Fine-tuning all layers typically outperforms training only the classifier head
3. The gap between transferred and randomly initialized features widens as the target task diverges from the source domain

2.11.2 Applications in Handwritten Text Recognition

While natural images differ significantly from document images, transfer learning from ImageNet has proven highly effective for handwritten text recognition [25].

. Lower layers of pretrained models capture fundamental visual features such as edge orientations, curvature detectors, and texture patterns—all of which are relevant for character recognition.

For handwritten Bangla text specifically, transfer learning offers several advantages:

- **Character structure**: Edge and curve detectors learned from natural images transfer well to detecting character strokes
- **Limited data availability**: With only thousands of samples per class, training from scratch would be impractical
- **Domain adaptation**: Fine-tuning adapts general features to the specific characteristics of handwritten Bangla script

2.11.3 Fine-Tuning Strategies

Several strategies exist for adapting pretrained models to target tasks [25]:

Feature extraction: The pretrained backbone is frozen, and only a new classification head is trained. This approach is computationally efficient and works well when the target domain is similar to the source domain.

Full fine-tuning: All layers of the pretrained model are updated during training. This approach allows maximum adaptation but requires careful regularization to prevent overfitting.

Progressive fine-tuning: A hybrid approach where later layers are trained with higher learning rates while early layers are updated more slowly, preserving general features while adapting task-specific ones.

In this thesis, both ViT-Base (pretrained on ImageNet-21k) and ConvNeXt-Tiny (pretrained on ImageNet-1k) are fine-tuned using a two-stage strategy: initial feature extraction (head-only training) followed by full fine-tuning of all layers. This approach balances adaptation with preservation of pretrained features.

2. 12 Overview of Ensemble Learning

Ensemble learning is a machine learning technique in which multiple models are combined to solve the same problem and improve predictive performance. Instead of relying on a single model, ensemble methods integrate the outputs of several individual models, often referred to as base learners or weak learners, to produce a more accurate and robust final prediction [26]. The fundamental idea is that different models may capture distinct patterns or aspects of the data, so combining their predictions reduces individual errors and improves generalization [27].

One key advantage of ensemble learning is its ability to reduce variance and bias and improve overall model stability [26]. Ensemble learning methods are generally categorized into sequential ensemble methods and parallel ensemble methods.

2.12.1 Sequential Ensemble Methods

Sequential ensemble methods train models one after another, where each new model attempts to correct the errors of previous models. Models are not independent; each focuses on improving mistakes made by earlier models [29]. Common sequential ensemble techniques include boosting and stacking.

2.12.1.1 Boosting

Boosting combines several weak learners into a strong predictive model by training them sequentially, with each subsequent model placing greater emphasis on samples misclassified by previous models [29]. The final prediction is a weighted combination of all models' predictions: The final prediction is a weighted combination of all models' predictions, defined in (5), where $F(x)$ is the final ensemble prediction, $h^m(x)$ is the prediction of the m-th weak learner, α^m is its weight, and M is the total number of models.

$$F(x) = \sum_{m=1}^{M} \alpha_m \ h_m(x) \tag{5}$$

2.12.1.1.1 Gradient Boosting

Gradient Boosting builds models sequentially by minimizing a loss function using gradient descent [30]. Instead of adjusting sample weights directly, it trains each new model to predict the residual errors of the previous model. Starting from an initial prediction $F_0(x)$, new models are added sequentially according to (6), where $F^m(x)$ is the updated model after iteration m, $h^m(x)$ is the new weak learner, and γ^m is the learning rate. Gradient boosting underpins algorithms such as XGBoost, LightGBM, and CatBoost [40].

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x) \tag{6}$$

2.12.1.1.2 AdaBoost

AdaBoost (Adaptive Boosting) adjusts sample weights based on classification errors [31]. Initially, all samples receive equal weights. After each iteration, misclassified samples receive higher weights so the next model focuses on challenging instances. The final prediction is a weighted majority vote defined in (7), where $h^m(x)$ is the prediction of the m-th classifier and α^m is its importance weight. AdaBoost has been widely applied in face detection and text classification.

$$F(x) = sign\left(\sum_{m=1}^{M} \alpha_m h_m(x)\right) \tag{7}$$

2.12.1.2 Stacking

Stacking (stacked generalization) trains multiple base models and combines their predictions using a meta-learner [32]. Several different models are trained on the same dataset, and their predictions are used as inputs for a meta-model. The meta-model learns how to combine the base predictions to produce the final output, defined in (8), where h_1, h_2, …, h^n are base models, g is the meta-learner, and y is the final prediction.

$$y = g\left(h_1(x),\ h_2(x),\ \ldots\ h_n(x)\right) \tag{8}$$

2.12.2 Parallel Ensemble Methods

Parallel ensemble methods train multiple models independently and simultaneously, without sequential dependency. The predictions are then combined to produce the final output [33]. Common parallel ensemble methods include bagging and voting.

2.12.2.1 Bagging

Bagging (Bootstrap Aggregating) improves model stability and reduces variance by training multiple models on different subsets of the training data [33]. Multiple training datasets are generated using bootstrap sampling (random sampling with replacement), each used to train an independent model. The final prediction is the average or vote across all models' predictions, defined in (9). A well-known example is the Random Forest algorithm, which combines multiple decision trees trained on different bootstrap samples [34].

$$F(x) = \frac{1}{M}\sum_{m=1}^{M} h_m(x) \tag{9}$$

2.21.2.2 Voting

Voting is a simple yet effective ensemble technique where multiple independently trained models combine their predictions to produce the final result [35]. Voting ensembles are divided into three types: Hard Voting, Soft Voting, and Weighted Soft Voting.

2.11.2.2.1 Hard Voting

Hard voting combines predictions by selecting the class that receives the majority vote among the models, defined in (10). For example, if three models predict Class A, Class B, and Class A respectively, the final prediction is Class A as it receives the majority of votes.

$$y = mode\big(y_1(x),\, y_2(x),\, \ldots\, y_M(x)\big) \tag{10}$$

2.11.2.2.2 Soft Voting

Soft voting uses the predicted probabilities of each class rather than predicted labels [35]. The probabilities from all models are averaged as in (10), and the class with the highest average probability is selected as in (12). Soft voting generally performs better than hard voting because it considers the confidence of each model.

$$P(y = c) = \frac{1}{M}\sum_{m=1}^{M} P_m(y = c \mid x) \tag{11}$$

$$y = arg\, max_c\, P(y = c) \tag{12}$$

2.11.2.2.3 Weighted Soft Voting

Weighted soft voting extends soft voting by assigning different weights to models based on their performance [36]. The weighted probability for each class is computed as in (13), where w^m is the weight assigned to the *m*-th model and $P^m(y = c \mid x)$ is its predicted probability for class *c*. The final prediction is the class with the highest weighted probability. Models with higher accuracy can be given larger weights, allowing them to contribute more to the final prediction [36]. In this research, predictions from different deep learning models are combined using hard voting and weighted soft voting strategies to improve classification performance.

$$P(y = c) = \frac{\sum_{m=1}^{M} w_m P_m(y = c \mid x)}{\sum_{m=1}^{M} w_m} \quad (13)$$

2.13 Dataset Construction in Low-Resource Settings

Constructing labeled datasets for handwriting recognition is labor-intensive, requiring the recruitment of writers, the collection of handwritten samples, their digitization, and the annotation of ground-truth labels. For low-resource languages or specialized domains, publicly available datasets are often scarce or non-existent [8].

Common strategies to address data scarcity include synthetic data generation (rendering handwriting using fonts and style transfer), few-shot learning, semi-supervised learning with pseudo-labeling, and aggressive data augmentation. In the context of Bangla HTR, the CMATERdb series [16] and the ISI Bangla databases are the most widely used benchmarks, but neither covers administrative place names or upazila-level granularity.

For the present task, 320 handwritten pages were collected from annotators spanning different ages and professions. Data augmentation—including random rotation, affine transformation, random erasing, and Mixup [38]—was applied to artificially expand the effective training set and introduce realistic variation. Approximately 10% of the segmented images required manual correction of the automatic segmentation to ensure label quality.

2.14 Research Gap and Motivation

A review of the existing literature reveals the following research gaps that motivate the present work:

Gap 1: Absence of a Dedicated Upazila-District Dataset

Despite the existence of multiple Bangla handwriting datasets [8, 14, 16, 17], no publicly available dataset exists for handwritten Bangla upazila–district pair recognition covering all 496 administrative pairs. Existing resources focus on isolated characters, document-level text, or district names alone, none addressing the paired administrative names essential for real-world applications such as postal automation and document digitization.

Gap 2: Limited Word-Level Recognition Research

Most existing Bangla handwritten text recognition systems focus on isolated characters or digits [6, 7, 10, 12], with word-level recognition for Bangla remaining significantly underexplored. The challenges of high inter-class similarity (shared name prefixes), high intra-class variability (diverse handwriting styles), and complex character structures have not been systematically addressed for administrative name recognition.

Gap 3: Unexplored Complementary Strengths of Modern Architectures

While Vision Transformers [4] capture global context through self-attention and ConvNeXt [5] excels at local pattern extraction through depthwise convolutions, their synergistic combination through ensemble methods remains unexplored for Bangla handwriting recognition. No prior work has systematically compared hard voting, stacking, and weighted soft voting for this domain or optimized fusion weights through validation grid search.

Gap 4: Missing Systematic Study of Segmentation and Training Strategies

A systematic investigation of segmentation and training strategies for multi-column handwritten pages containing paired administrative names is lacking in the literature. This includes evaluation of segmentation pipeline design, activation function choices (ReLU vs. GELU) [22], two-stage training protocols [23, 24], and balancing augmentation techniques [38] for class-imbalanced datasets.

This thesis directly addresses these gaps by constructing a novel dataset, developing a complete end-to-end recognition pipeline, and providing thorough experimental analysis using evaluation metrics.

CHAPTER 3: RESEARCH METHODOLOGY

3.1 Overall Working Process of Bangla Upazila–District Pair Name Recognition

This chapter discusses the methodology of the research work. To develop an end-to-end system for Bangla Upazila–District Pair Name Recognition, the complete pipeline is divided into three sub-problems:

1. Dataset collection,
2. Text segmentation,
3. Data Preprocessing
4. Classification.

Figure 3.1 demonstrates the process of creating the BNHW-UDPNR dataset (left), the overall architecture for word segmentation (middle), and the steps of the proposed model (right).

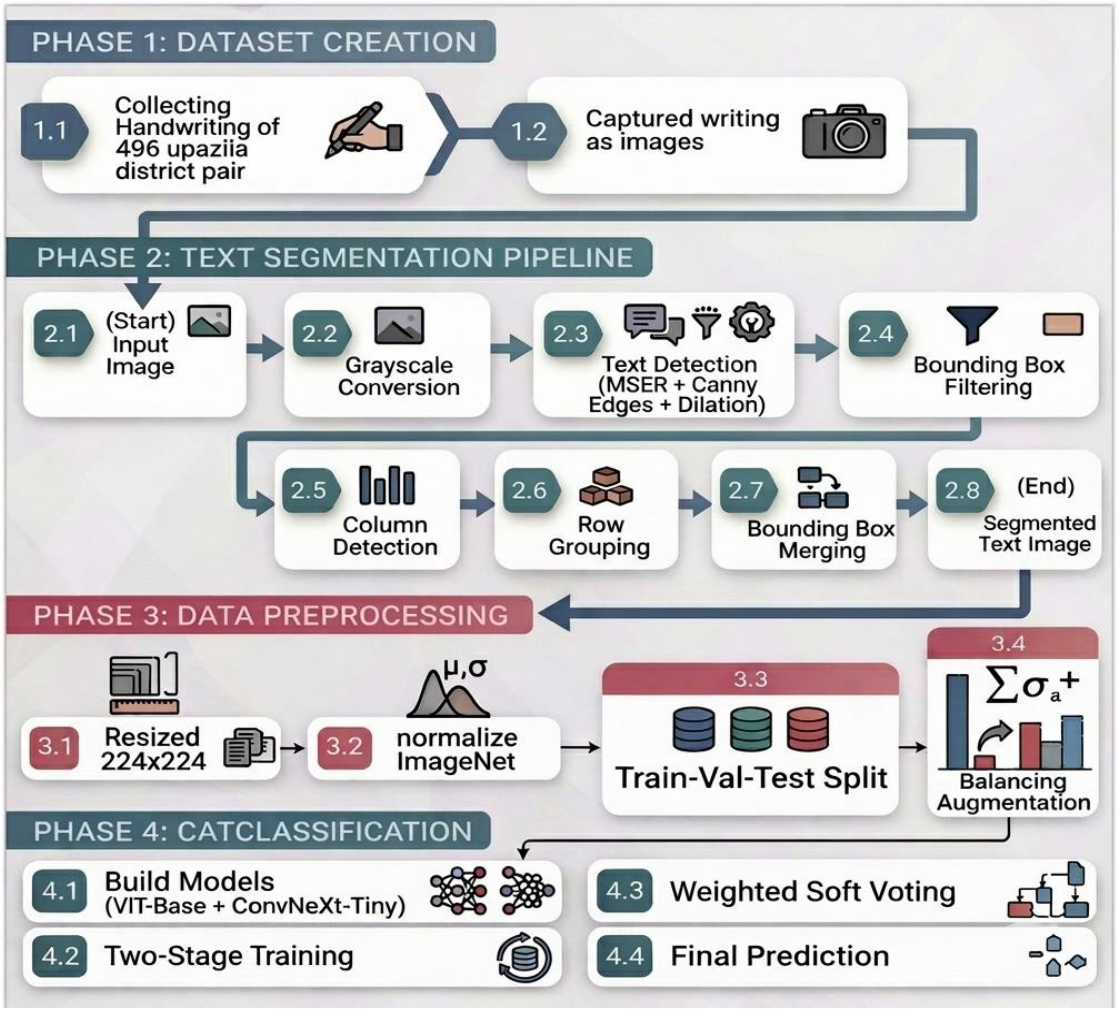


Figure 3.1: Overall working process of Bangla Upazila–District name recognition.

3.2 Dataset Collection

The dataset was created from handwriting samples representing 496 unique Bangla handwritten upazila–district pairs. For each pair — for example, বাগেরহাট, চিতলমারী — multiple individuals of various ages wrote the name on plain paper, introducing natural variations in handwriting style, size, and skew. The pages included both colored and white paper. We also collected some postal envelopes and forms for the dataset. These collected pages are included in the dataset and are ready for segmentation in the next step. The handwritten sheets were then digitized using a high-resolution camera, producing RGB images. These digitized images serve as the raw input for the subsequent text segmentation phase.

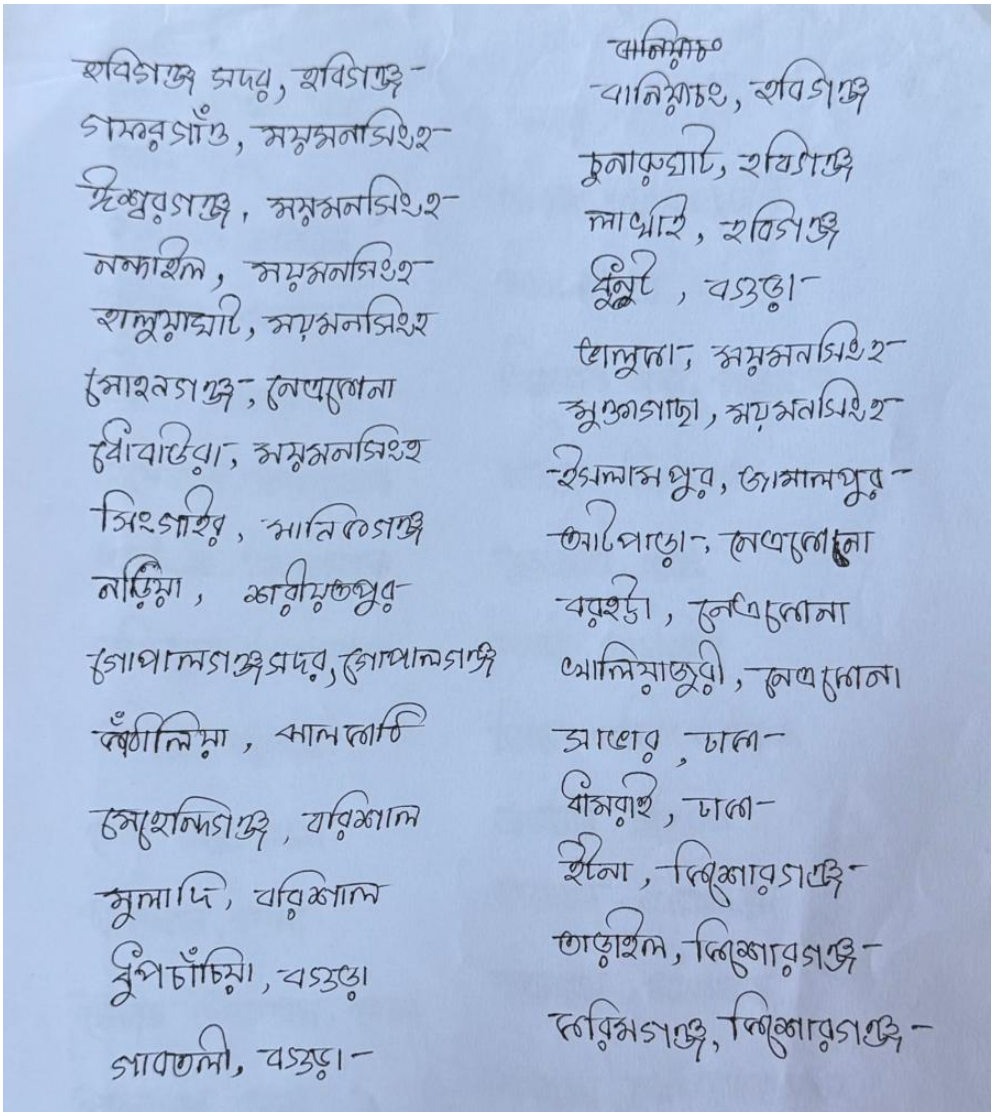


Fig. 3.2: Sample handwritten image.

3.3 Text Segmentation

After acquiring the handwritten images, several preprocessing steps were applied for word segmentation. First, the input image was converted to grayscale to simplify further processing. Two complementary text-detection methods were then used to identify candidate text regions.

- **MSER Detection:** MSER detection to find stable character-level regions [30]
- **Canny Edge Detection and Dilation:** Use Canny edge detection to identify edges [30], then apply morphological dilation to connect nearby edges into larger line components [28]

Bounding boxes from both methods were collected and filtered by size to remove obvious non-text areas. Column splitting was detected using a vertical projection profile, separating boxes into left and right groups for two-column documents. Within each column, boxes were grouped into rows based on vertical position. Adjacent boxes were merged when they were either close horizontally or overlapped significantly, and a tight rotated rectangle was computed around each merged group. Finally, each rotated rectangle was rectified and cropped, producing segmented text images containing individual upazila–district pairs. The dataset creation and word segmentation processes will be described in the next chapter, Dataset Collection, Preprocessing, and Segmentation.

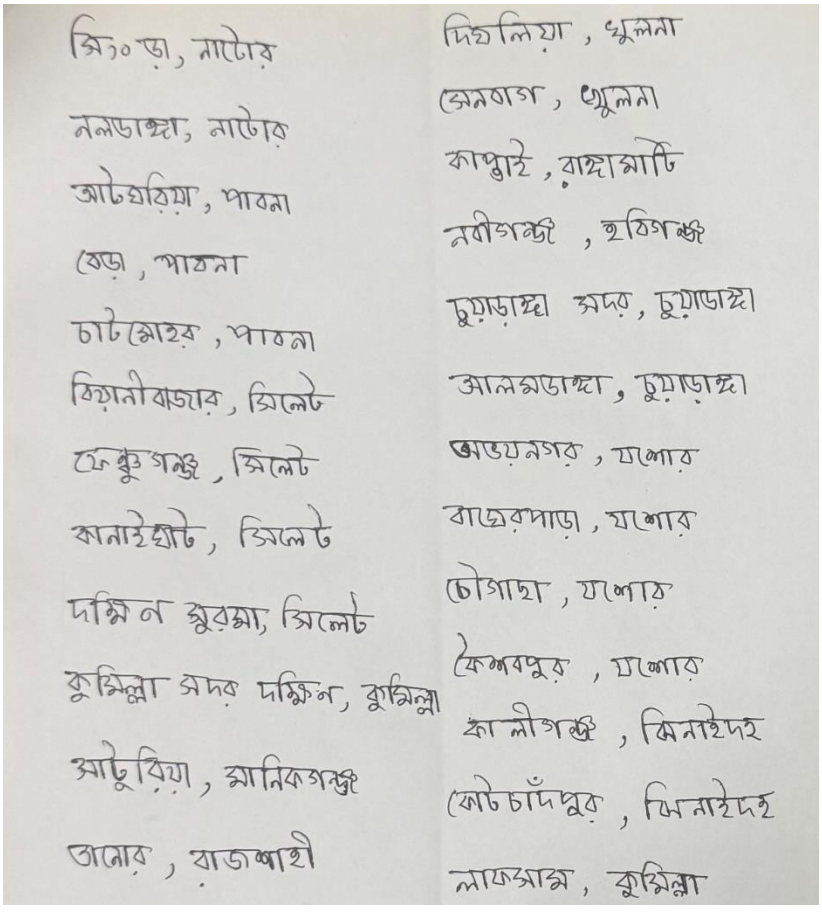


Figure 3.3: Input Image

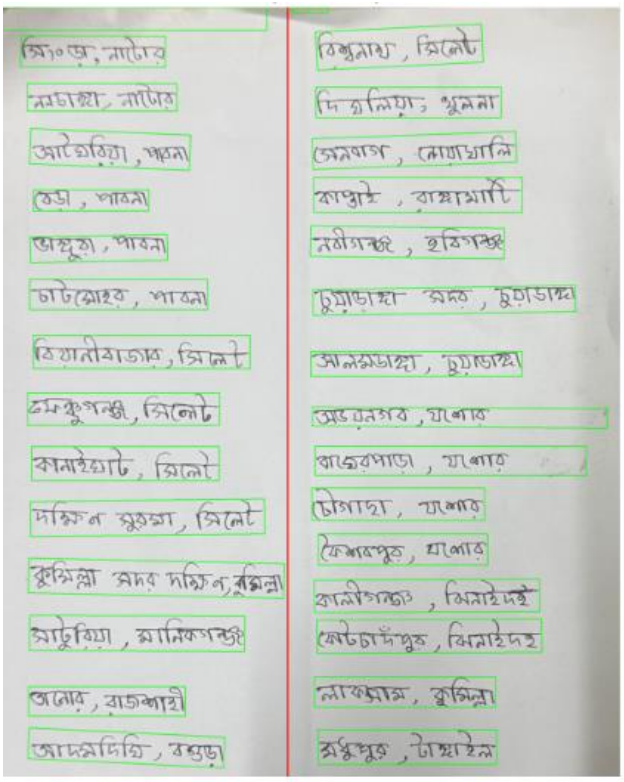


Figure 3.4: Bounding Box



Figure 3.5: Segmented Images.

3.4 Data Preprocessing

After segmentation, the cropped images were resized to 224×224 pixels using bilinear interpolation and normalized with ImageNet mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$. The dataset was split into training (70%), validation (20%), and test (10%) sets using a deterministic, stratified split (seed 42); classes with fewer than 5 samples were removed to ensure statistical reliability.

To address class imbalance, a balancing augmentation strategy was applied exclusively to the training set. Each under-represented class was augmented to match the size of the largest class by generating synthetic samples through random combinations of 3–6 operations from a pool that includes geometric (affine, perspective, elastic), photometric (brightness/contrast, noise, blur, JPEG), and document-specific (ink dropout, ink bleed, uneven light, stains, background dots) transformations. A special TextAug operation chains several of these to simulate realistic document degradations. Duplicates were prevented by computing a difference hash (dHash) for each generated image and discarding any sample whose hash already existed in the class. The full data preprocessing pipeline, including the automatic segmentation steps, is detailed in Chapter 4.

3.5 Classification

After acquiring the segmented dataset, all images were resized to a fixed size of 224×224 pixels to ensure uniformity. Image augmentation was then performed to increase the diversity of the training data, including rotations, affine transformations, translations, random erasing, and brightness adjustments.

Two deep learning models were developed: Vision Transformer (ViT-Base) and ConvNeXt-Tiny. Both models were trained using a two-stage approach. In the first stage, only the custom classification head was trained while the backbone remained frozen. In the second stage, the entire network was fine-tuned. The AdamW optimizer was used with cross-entropy loss and label smoothing, and dropout layers were added to the classification head to minimize overfitting. Final predictions from both models were combined through weighted soft voting to enhance overall classification accuracy.

3.6 Tools Requirement

- PyTorch
- NumPy
- Matplotlib
- TensorFlow
- Pandas
- Keras
- Scikit-learn
- Skimage
- SciPy
- io
- OS
- OpenCV

CHAPTER 4: TEXT SEGMENTATION

In handwritten text recognition research, the quality of preprocessing directly impacts the effectiveness of the learning model. Unlike printed text, handwritten data naturally exhibits substantial variations in writing style, stroke thickness, alignment, illumination, background texture, and noise. Therefore, building a robust preprocessing framework is not optional — it is crucial.

This chapter outlines the complete data preparation process for the BNHW-UDPNR (Bangla Handwritten Upazila–District Pair Name Dataset). It covers everything from collecting raw handwritten pages to creating a final dataset ready for deep learning models such as Vision Transformer (ViT) and ConvNeXt.

The dataset has been made publicly available on Kaggle.[1]

4.1 BNHW-UDPNR: Data Annotation Process

This section will provide a detailed description of our data annotation approach. The BNHW-UDPNR dataset was created specifically to facilitate supervised classification of Bangla handwritten Upazila–District pairs. The entire workflow — from text preparation to the final cropped labeled images — is shown in Figure 4.1.

[1] https://www.kaggle.com/datasets/d22323mojahidulislam/upazila-district-final-v2

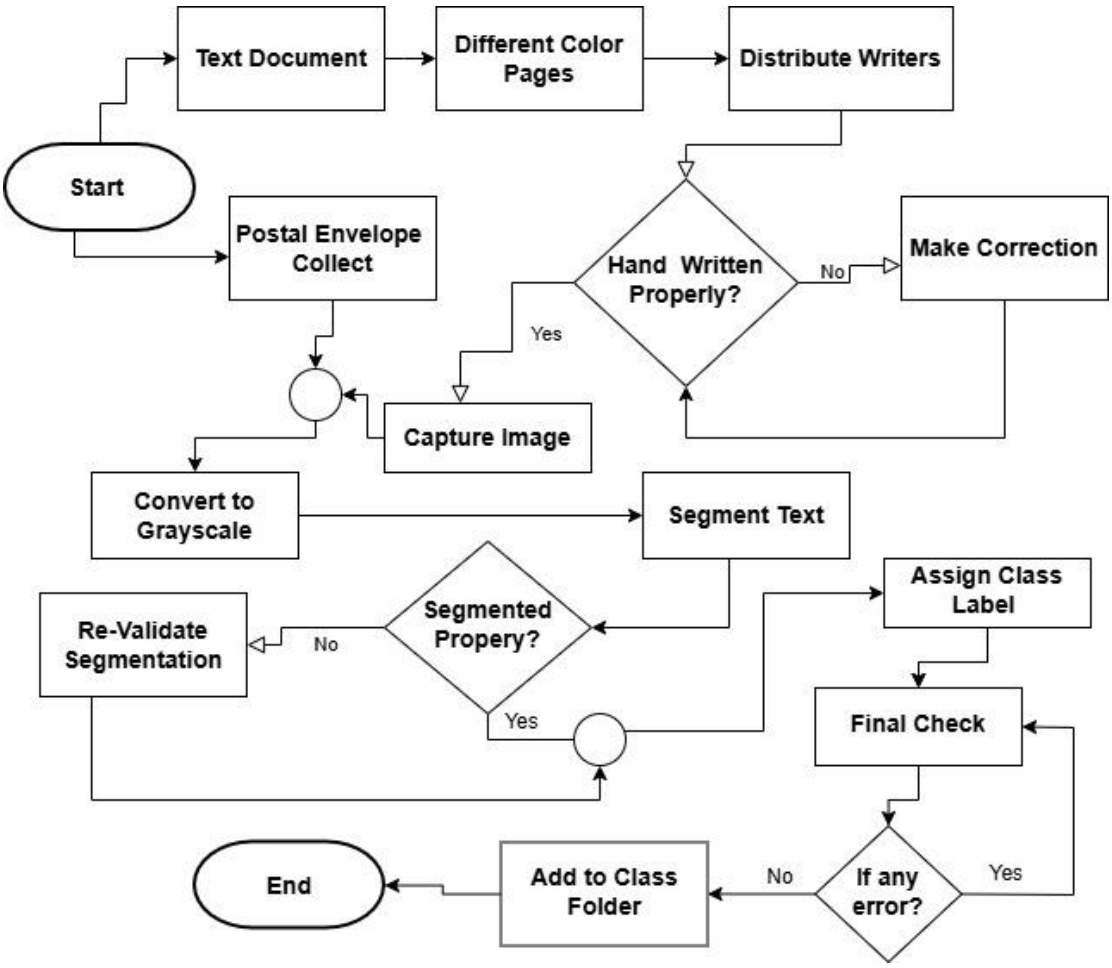


Figure 4.1: Flowchart of the complete annotation and preprocessing pipeline.

The annotation process consists of three main stages:

1. Data source preparation
2. Handwritten sample collection
3. Digitization and Labelling

4.1.1 Data Source and Collection

Data annotation involves labelling information so that machines can learn from it. It is especially valuable for supervised machine learning (ML), where a system uses labelled datasets to analyze, interpret, and generalize input patterns to generate desired outputs. Initially, a text file was created containing 496 upazila–district pairs from Bangladesh. Blank pages and the source file were given to people of various ages. Each person wrote 25 to 30 upazila–district pairs in two columns on the blank pages. The finished hard copies were then collected from the annotators.

4.1.2 Postal Envelope and Form Collection

As part of the dataset, postal envelopes were collected and photographed. Only the upazila–district name was extracted from each envelope through manual segmentation. Postal envelope images were primarily collected for testing purposes.

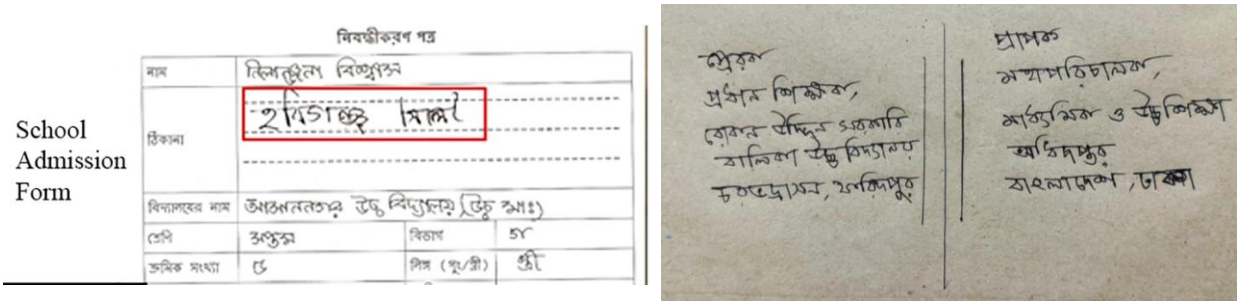


Fig. 4.2: Sample images of a School Admission Form and Postal Envelope.

4.1.3 Data Distribution

Data was distributed among individuals of different ages and professions for annotation. Specifically, 60% of the data was prepared by students, while the remaining 40% was created by teachers, officials, and other individuals. A total of 320 completed pages were collected from participants.

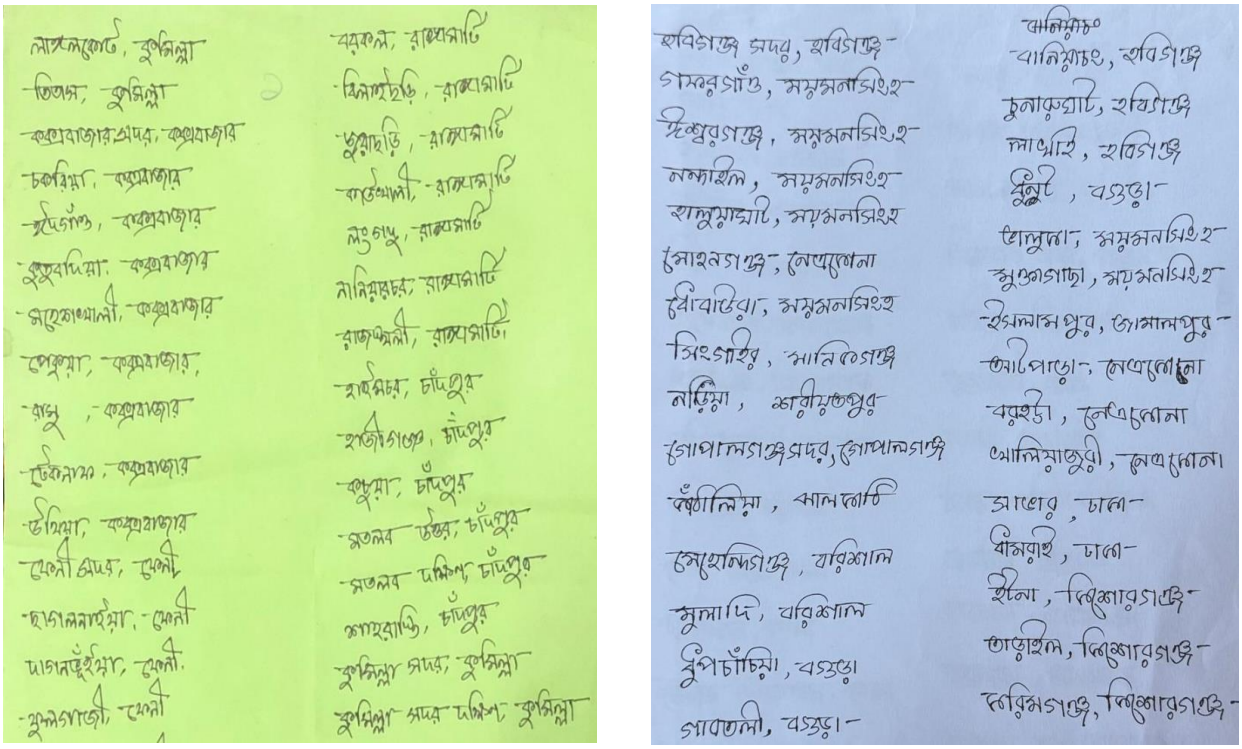


Figure 4.3: Sample page of a handwritten dataset.

Images of the collected hard copies were then captured and stored in a folder. Each image was converted to grayscale, and an automated text segmentation pipeline was applied to crop individual upazila–district-pair regions, yielding 9808 segmented images. Following segmentation, 496 folders were created, each named after a specific upazila–district pair, and segmented images were manually saved into the corresponding folders. All images within each folder were automatically renamed using the pattern {class_name}_{counter:04d}.png for consistency.

4.2 Automatic Text Segmentation

An unsupervised approach was used to segment the text regions. The segmentation mechanism is composed of the following steps:

Pre-processing:

1. RGB to Grayscale Conversion
2. Thresholding
3. Edge Detection
4. Morphological Operation and Noise Removal

Segmentation Process:

- Column Detection
- Row Grouping
- Connected Component Analysis (CCA)
- Bounding Box Merging with minAreaRect
- Text Cropping

4.2.1 Pre-processing

Pre-processing typically simplifies the input so it can be processed by the system. Figure 4.4 illustrates the pre-processing steps that have been used.

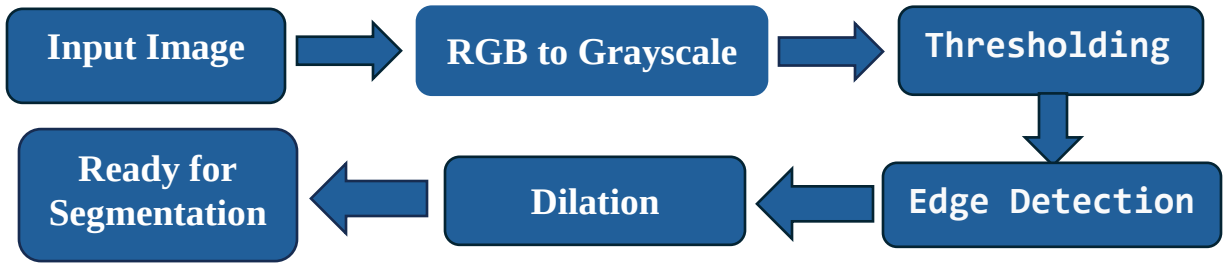


Figure 4.4: Pre-processing steps.

4.2.1.1 RGB to Grayscale Conversion

The input image is read as an RGB image with three channels: Red, Green, and Blue. Two common methods exist for this conversion. The Average method computes the mean of R, G, and B:

Grayscale = (R + G + B) / 3

The Weighted (Luminosity) method assigns different weights to each channel according to human visual perception:

Grayscale = 0.299R + 0.587G + 0.114B

OpenCV reads images as BGR rather than RGB. The cvtColor function is used to convert the BGR image to grayscale:

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

4.2.1.2 Thresholding

Thresholding is a technique for image segmentation that transforms a grayscale image into a binary image with only two intensity levels (0 or 1). If a pixel value exceeds the threshold, it is set to white; otherwise, it is set to black. This work uses the local Otsu's method for binarization.

Otsu's Technique: Thresholding separates foreground (1) pixels from background (0) pixels. Otsu's method is a variance-based technique that tests all possible threshold values, measuring the distribution of background and foreground pixels, and chooses the threshold that minimizes intra-class variance.

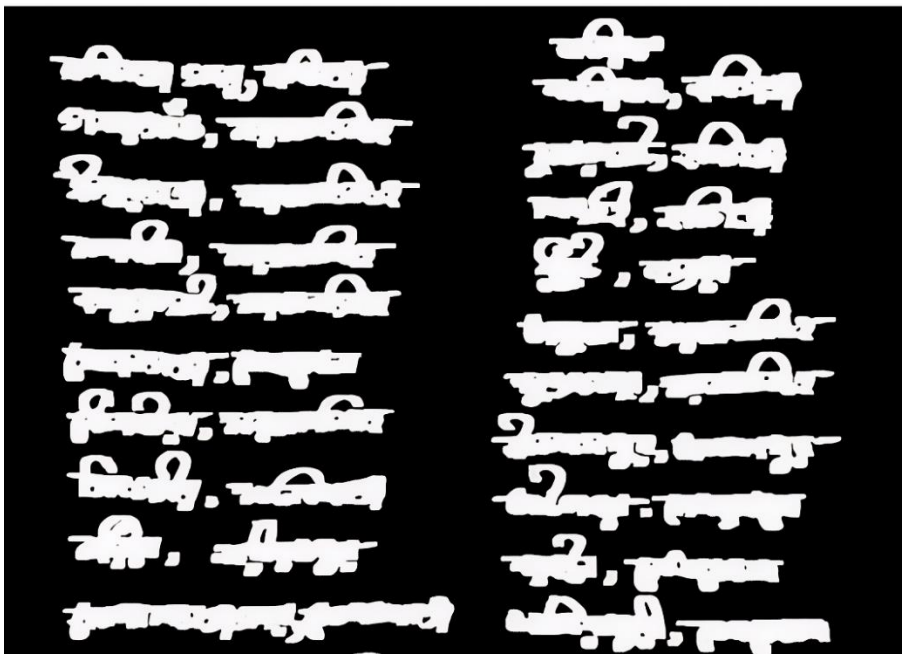


Figure 4.5: Thresholded images.

4.2.1.3 Edge Detection

Edge detection identifies points in a digital image where brightness changes rapidly. Canny edge detection extracts important structural details from visual objects while greatly reducing the volume of data that needs to be processed.

General criteria for edge detection include:

5. Edge detection must have a low error rate, capturing as many of the image's true edges as possible.
6. Detected edge points should accurately locate the edge's centre.
7. A specific edge in the image should only be marked once, and image noise should not create false edges.

Two complementary text-detection methods are employed:

- **MSER:** MSER (Maximally Stable Extremal Regions): Character-level blobs are detected with parameters delta=5, min_area=150, and max_area=10000. Bounding boxes with a width between 10 and 800 pixels and a height between 10 and 250 pixels are retained.
- **Canny + Dilation:** Canny edge detection + Dilation: Edges are extracted using Canny thresholds 50 and 150, followed by dilation with a 9×3 kernel (2 iterations) to connect characters into line-level components. Contours are then extracted, and bounding boxes with a width of 15–900 pixels and a height of 12–250 pixels are kept.

The union of MSER-based and Canny-based boxes provides a rich set of candidates covering both character-level and line-level structures.

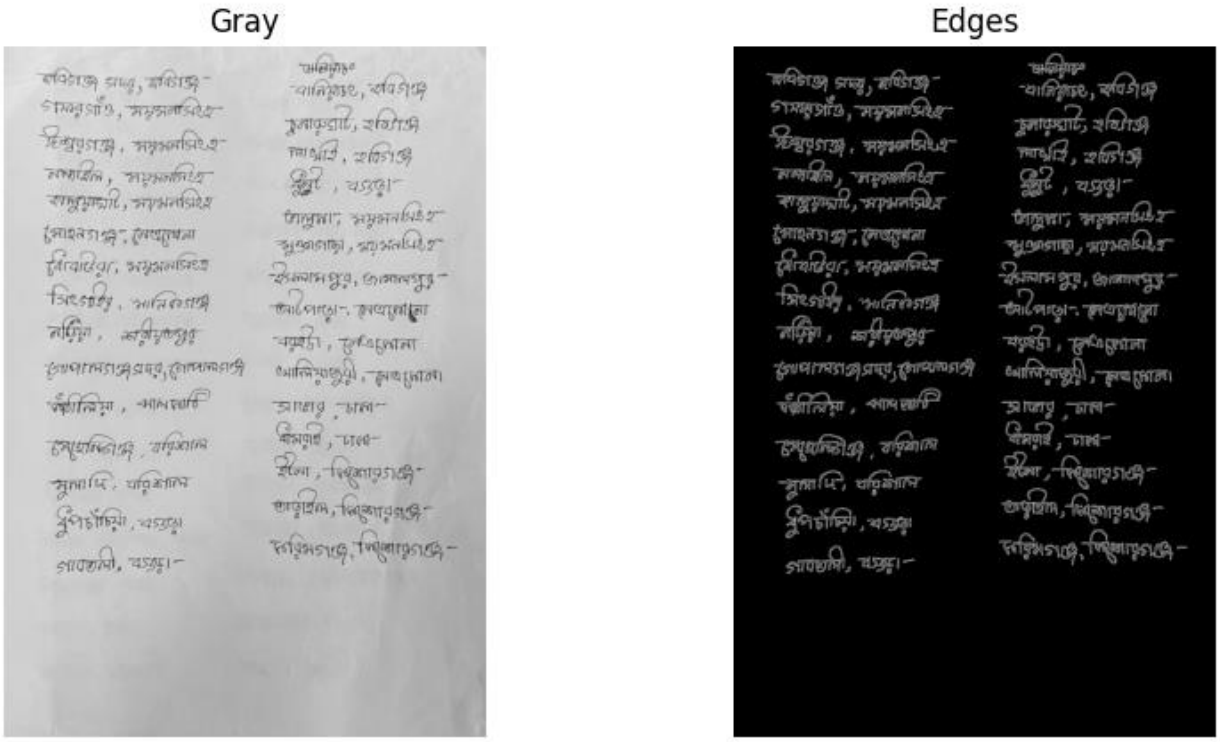


Figure 4.6: Edge detection images.

4.2.1.4 Morphological Operation and Noise Removal

Morphological operations refine binary images by removing noise and enhancing text structures. The two primary operations are dilation, which expands the boundaries of foreground objects, and erosion, which removes pixels from object boundaries.

First, morphological opening (erosion followed by dilation) is applied using a 3×3 kernel, effectively removing salt-and-pepper noise while maintaining the overall shape of text regions. Then, dilation with a larger 9×3 rectangular kernel connects nearby text components horizontally—particularly important for Bangla script, where characters within a word can be spaced apart. The result is a clean binary image with clearly defined text regions ready for connected component analysis.

4.2.2 Segmentation Process

After pre-processing, the image undergoes a series of segmentation steps to extract individual text regions.

4.2.2.1 Column Detection

Most raw data pages have two columns of text. A vertical projection profile is created after adaptive thresholding, and the global minimum between one-third and two-thirds of the image width is used as the column split point. Boxes are assigned to the left or right column based on their center x-coordinate.

4.2.2.2 Row Grouping

Boxes in each column are sorted by their top-left y-coordinate and grouped into rows based on vertical proximity. A box is added to an existing row if its vertical center is within 90 pixels of that row's average center.

4.2.2.3 Connected Component Analysis

Connected Component Analysis (CCA) is a fundamental technique for detecting and labeling distinct connected regions in binary images. After pre-processing, the binary image contains multiple disconnected regions corresponding to different characters, words, or lines. CCA identifies each distinct region and assigns a unique label to it.

Theoretical Foundation: In graph theory, a connected component is a maximal set of pixels in which any two pixels are connected by a path of adjacent pixels. For binary images, connectivity is defined in two ways:

- **4-Connectivity:** 4-connectivity: A pixel is connected to its four orthogonal neighbors (up, down, left, right).
- **8-Connectivity:** 8-connectivity: A pixel is connected to its eight neighbors, including diagonals.

In our implementation, we use 8-connectivity because the Bangla script frequently features diagonal strokes and compound characters that orthogonal connectivity alone cannot fully capture. This choice ensures that characters with diagonal connections (e.g., র and য) are correctly identified as single components.

Implementation: OpenCV's cv2.connectedComponentsWithStats() function implements a two-pass algorithm and returns the total number of components, a label matrix where each pixel contains its component label, and statistics for each component, including bounding-box coordinates and area.

Bounding Box Extraction: For each component (excluding background), we extract its bounding box from the statistics. These boxes are represented as (x, y, w, h), where (x, y) is the top-left corner.

Filtering: Components with an area smaller than 50 pixels are discarded as noise. This threshold was empirically determined based on the typical size of handwritten Bangla components in our scanned images (300 dpi). Additional filters, such as aspect ratio or solidity, may optionally be applied to remove non-text artifacts, but the area threshold alone proved sufficient for our dataset.

Role in Pipeline: The output is a set of initial bounding boxes representing candidate text regions at the character, word, or line level. These boxes serve as the foundation for the subsequent merging step (Section 4.2.2.4), where they are combined horizontally within rows to form complete text regions corresponding to entire upazila–district pairs. Figure 4.7 illustrates the result of CCA on a sample image.

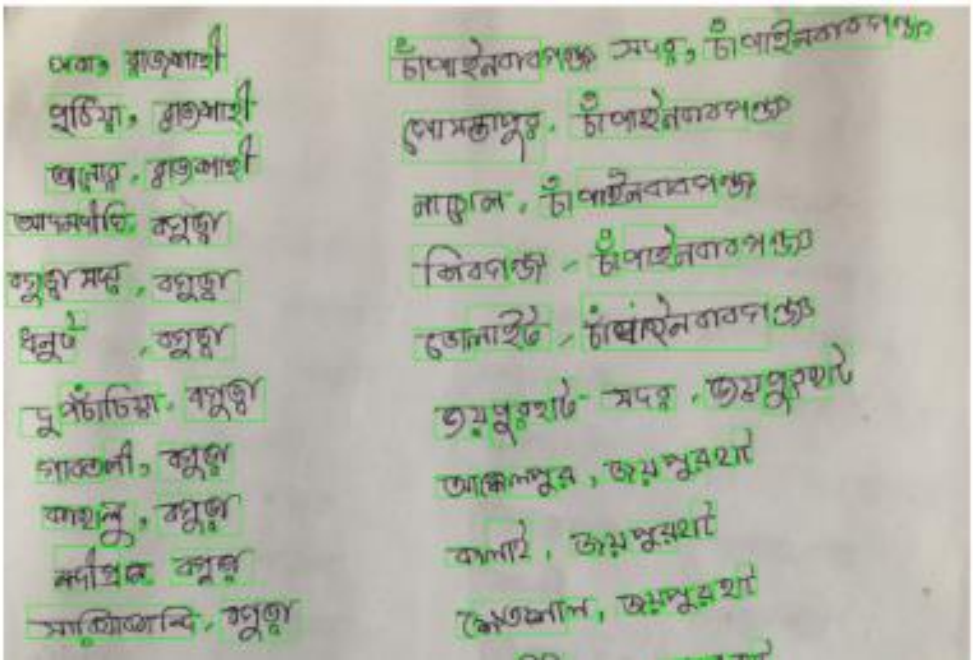


Figure 4.7: Connected component analysis with Bounding Boxes.

4.2.2.4 Bounding Box Merging with minAreaRect

After grouping candidate text boxes into rows, the next step is to merge adjacent boxes within each row to form a single bounding region enclosing an entire upazila–district pair. For each row, boxes are iterated in x-coordinate order. Two boxes are merged if they satisfy either of the following criteria:

- **Horizontal Proximity:** The gap between the right edge of the current box and the left edge of the next box is ≤ 35 pixels.

- **Vertical Overlap:** Vertical overlap: The vertical overlap between the two boxes exceeds 45% of the height of the smaller box.

When a merge occurs, the current merged box expands to encompass all constituent boxes by updating its minimum and maximum coordinates. This process continues until no further merges are possible within the row.

Once a group of boxes is finalized, we collect all four corner points of every box in the group and pass them to OpenCV's minAreaRect function. This function computes the minimum-area rotated rectangle that encloses all points. This is particularly useful for handwritten text, which may be skewed or curved. The function returns a rectangle defined by its Centre (x, y), Width w and height h, and Rotation angle θ (in degrees).

The resulting rotated rectangle provides a tight, accurate bounding region that follows the natural orientation of the handwritten text, unlike axis-aligned boxes which may include excess background or clip parts of the text. These rectangles are then passed to the text cropping stage (Section 4.2.2.5).

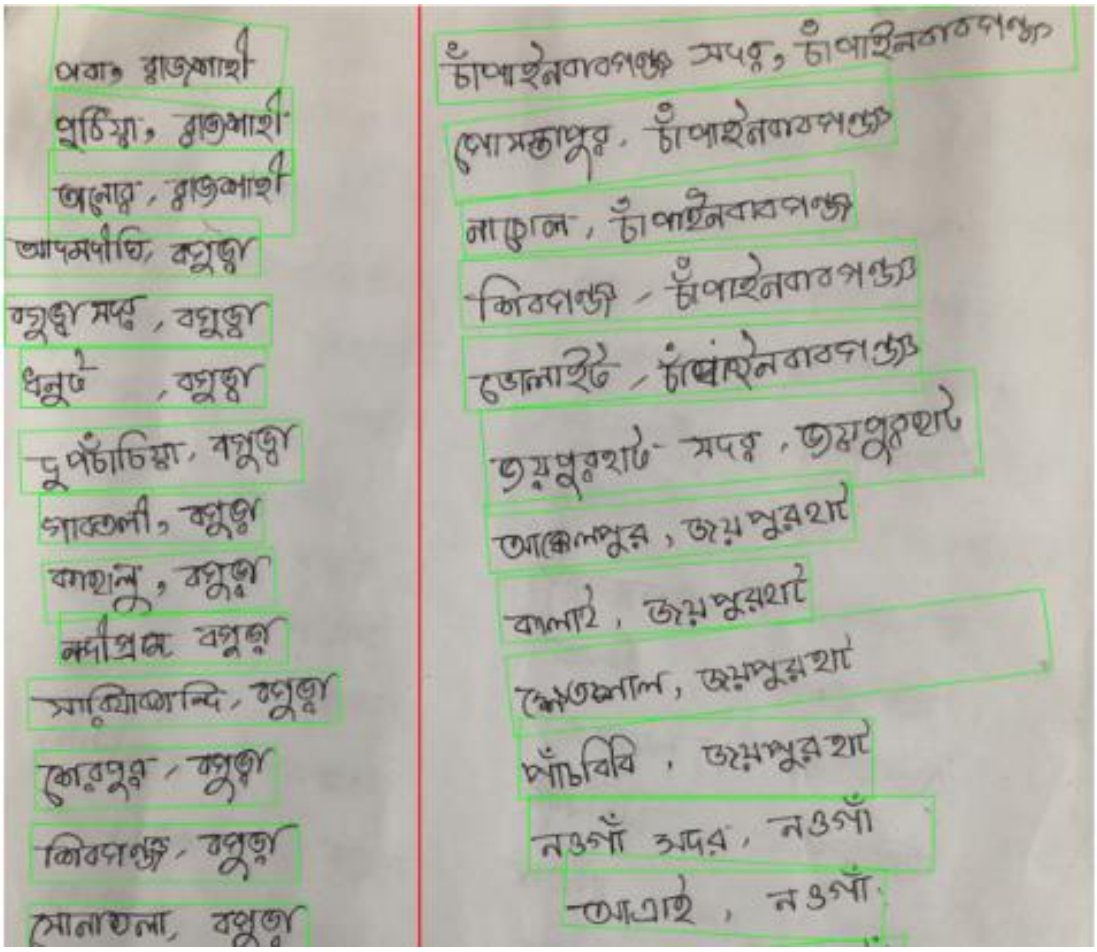


Figure 4.8: Bounding Boxes.

4.2.2.5 Text Cropping

After obtaining the rotated rectangle for each merged text region, the final step is to extract a clean, horizontally aligned image containing the upazila–district pair. For each rotated rectangle

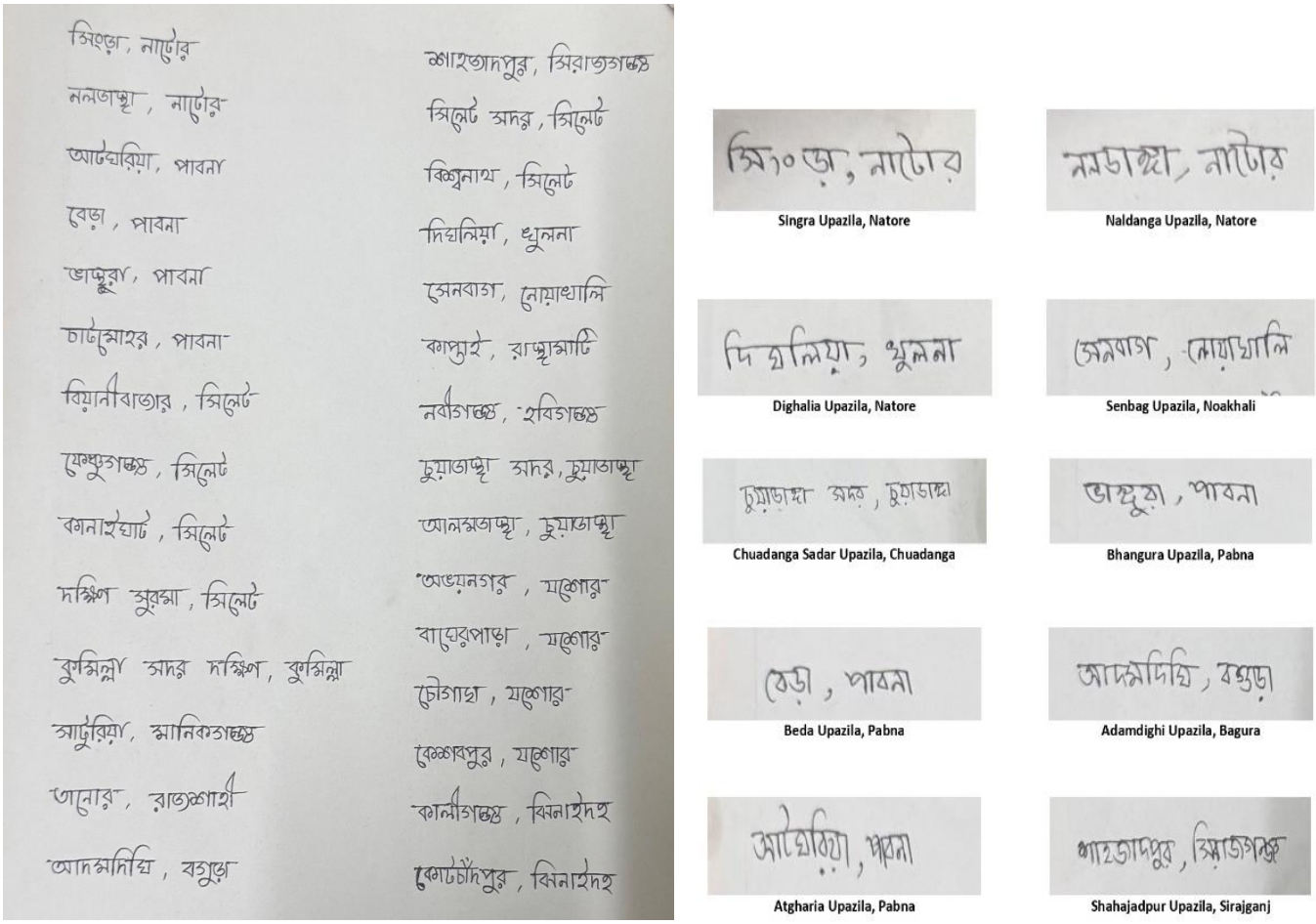
defined by its center (x, y), dimensions (w, h), and rotation angle θ, an **affine transformation** is applied to correct any skew. The entire image is warped using cv2.warpAffine with border replication to rotate the text horizontally.

The axis-aligned bounding box coordinates for cropping are calculated as:

$$x1 = centre_x - w/2,\ \ y1 = centre_y - h/2$$

$$x2 = centre_x + w/2,\ \ y2 = centre_y + h/2$$

Optional padding can be added to the bounding box to ensure that no part of the text — such as ascenders, descenders, or diacritics — is accidentally clipped. Figure 4.9 shows examples of original grayscale images alongside their corresponding segmented and cropped text regions.



(a) **(b)**

Figure 4.9: (a) Original grayscale image (b) Segmented and cropped text regions.

These cropped images constitute the final dataset used for training and evaluation during the classification stage.

4.3 Observation of Datasets

After applying the automatic text segmentation pipeline, varying degrees of success were observed depending on the quality of the original handwritten samples. Two representative cases are presented below.

Case 1: Successful Automatic Segmentation

Images with good contrast, clear handwriting, and minimal noise were segmented successfully. The bounding boxes tightly enclosed each upazila–district pair without clipping any text elements. Figure 4.10 shows an example of successful automatic segmentation.

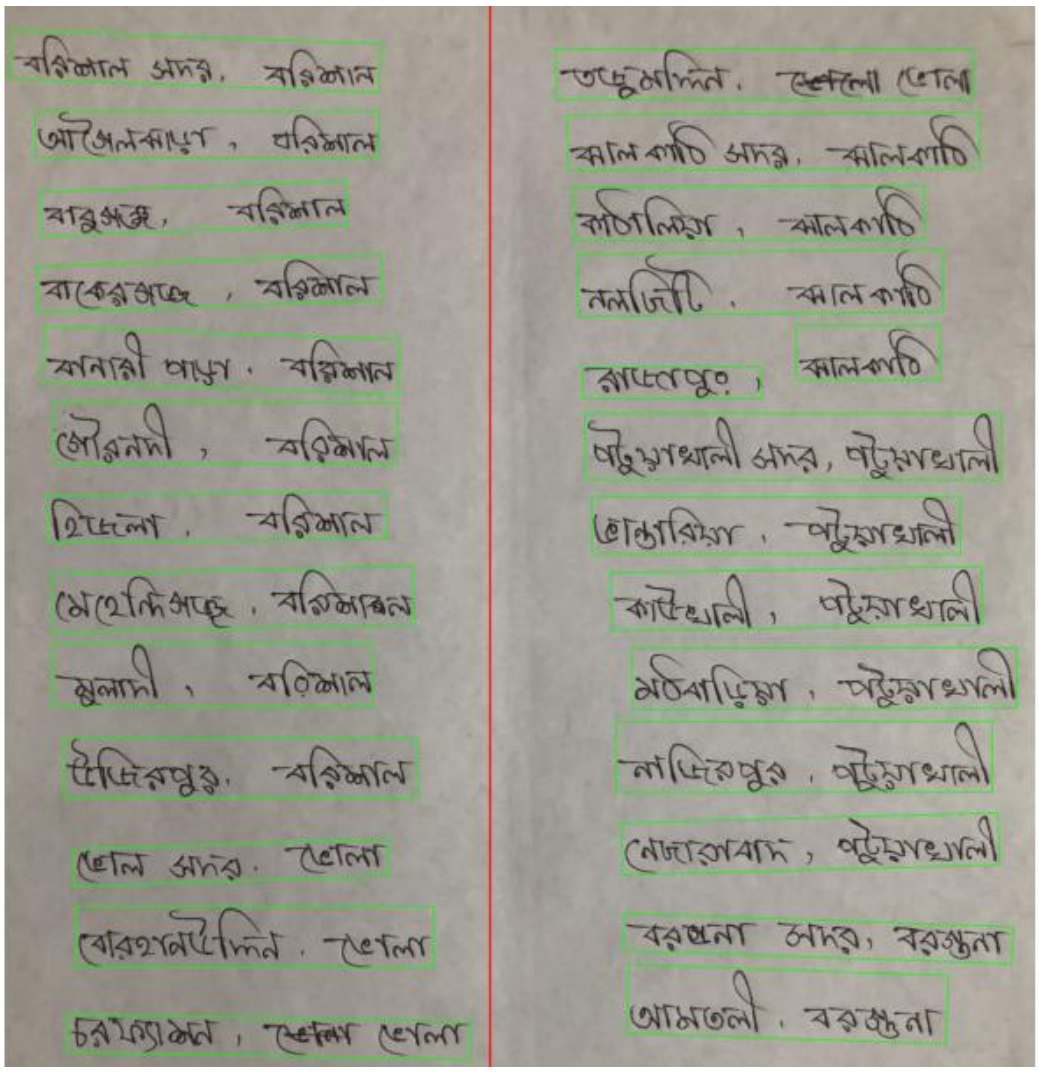


Figure 4.10: Successful automatic segmentation — text regions accurately detected and cropped.

The automatically segmented and cropped images need to be saved by selecting the appropriate Upazila-district pair level in their folder. After clicking "Save and Next", the images are saved to the folder.

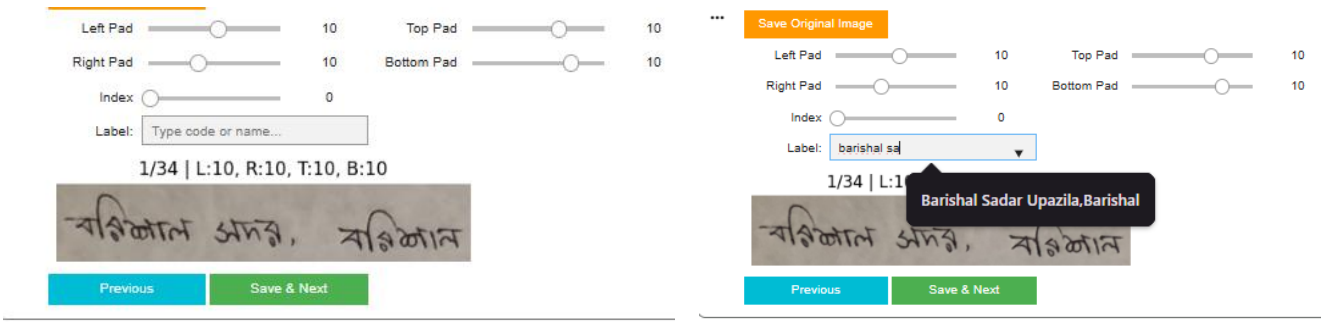


Figure 4.11: Example of saving automatically segmented and cropped image.

Case 2: Segmentation Requiring Manual Adjustment

Images with poor contrast, uneven lighting, ink bleeding, or faint handwriting could not be processed automatically. In these cases, the pipeline produced incomplete or inaccurate bounding boxes. Figure 4.12 shows a challenging image that requires manual intervention.

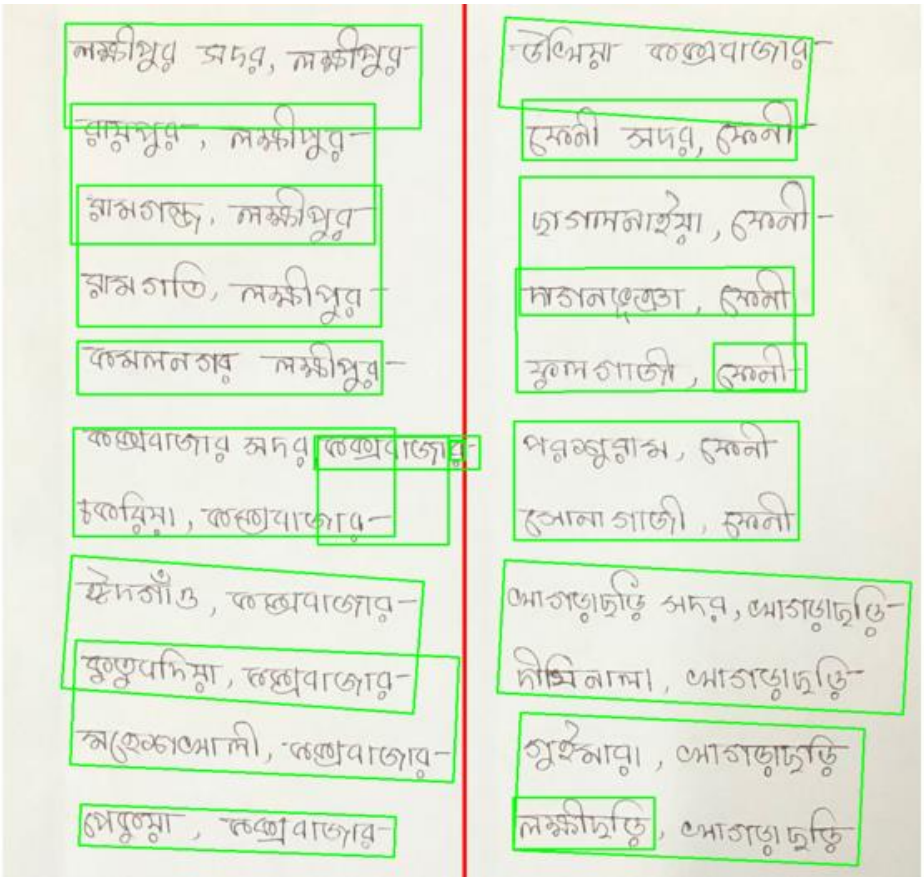


Figure 4.12: Challenging image requiring manual intervention — text regions inaccurately detected.

If the bounding box does not locate the correct position, use the toggoler to adjust it and crop the image.

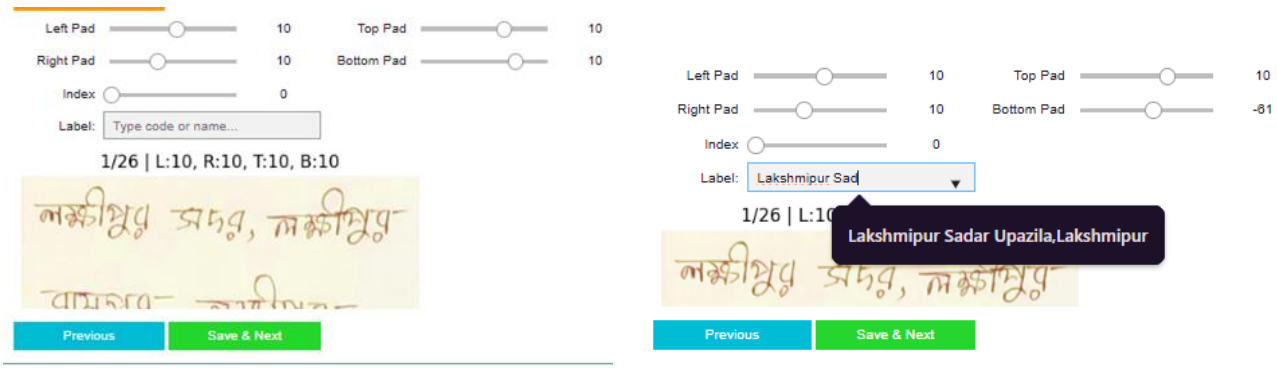


Figure 4.13: Example of adjusting the bounding box and saving a challenging image.

Approximately 90% of images were segmented automatically, while 10% required manual adjustments to ensure all text areas were precisely isolated for the final dataset.

4.4 Structure of our dataset:

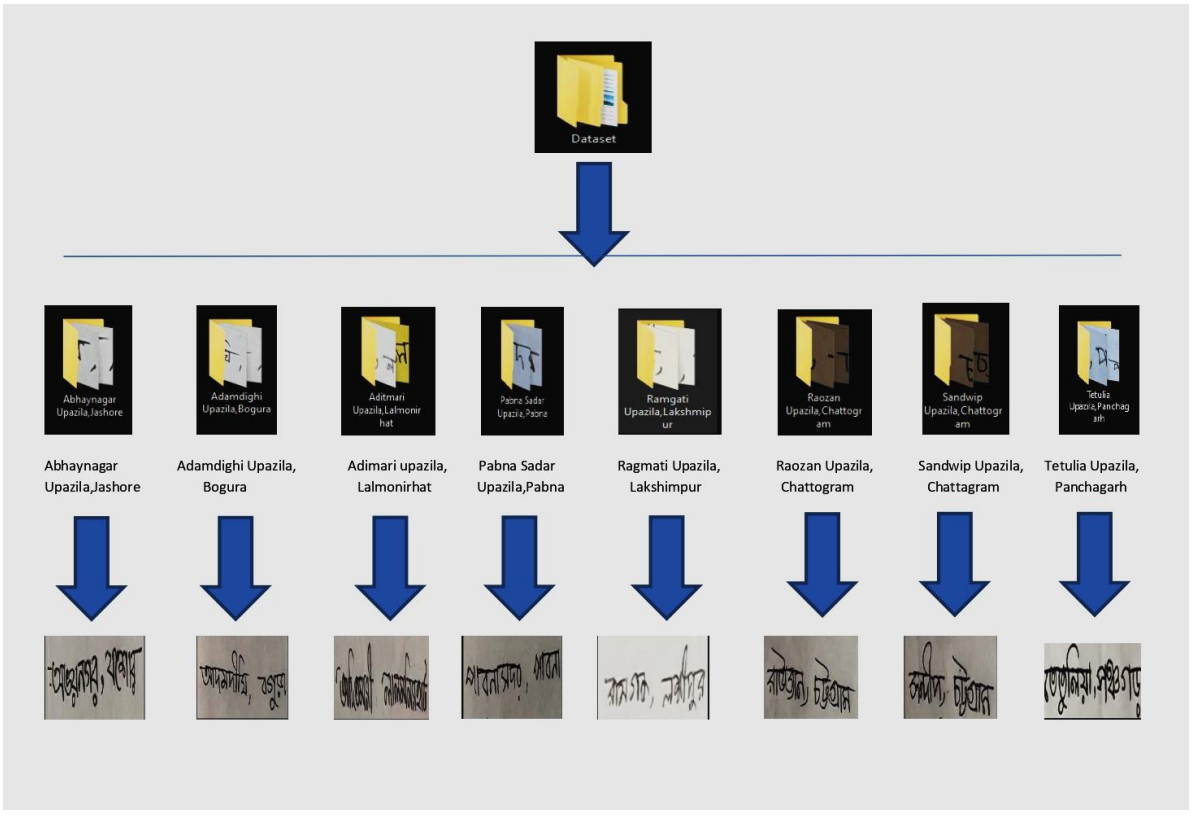


Figure 4.14: Structure of the Dataset

Figure 4.14 shows an example of the dataset folder organization, where samples are stored in subfolders named after their respective class labels. The folder named "496" contains data for the 496th class.

CHAPTER 5: DATA PREPROCESSING

Before being fed into the deep learning models, all input images undergo a series of preprocessing and augmentation steps to ensure consistency, improve generalization, and artificially expand the training dataset.

5.1 Image Preprocessing

The raw segmented images obtained from the text segmentation pipeline (Chapter 4) vary in size, contrast, and quality. To prepare them for model training, the following preprocessing steps are applied:

Resizing: All images are resized to a fixed dimension of 224 × 224 pixels. This size was chosen because it is the standard input size for both ViT-Base and ConvNeXt-Tiny models pretrained on ImageNet. Resizing is performed using bilinear interpolation, preserving the aspect ratio by first resizing the shorter side to 224 and then center-cropping to obtain a square image. This approach minimizes distortion while retaining the essential text content.

Normalization: After resizing, pixel values are normalized using the mean and standard deviation of the ImageNet dataset, as both models were pretrained on ImageNet. The normalization parameters are: Mean [0.485, 0.456, 0.406] and Standard deviation [0.229, 0.224, 0.225]. Normalization is applied consistently to training, validation, and test sets.

5.2 Data Augmentation

To improve model generalization and reduce overfitting, a variety of data augmentation techniques are applied to the training images. Augmentation artificially increases the diversity of the training set by introducing realistic variations that the model may encounter during inference. The augmentations are implemented using torchvision transforms and are applied on-the-fly during training.

Geometric Augmentations:

- Random Rotation: Images are rotated by a random angle within ±5°. This simulates slight skew variations that commonly occur in scanned handwritten documents.
- Random Affine Transformation: Translation of up to 3% of the image dimensions and shear of up to 5° are applied. This helps the model become invariant to small positional shifts and distortions.

Photometric Augmentations:

- Random Erasing: With probability 0.25, a rectangular region of the image is randomly selected and masked out by replacing its pixels with random values. The erased region has a scale between 0.02 and 0.08 of the image area and an aspect ratio between 0.3 and 3.3. This technique encourages the model to focus on multiple discriminative features rather than relying on a single region.

Advanced Augmentation — Mixup: In addition to basic augmentations, we employ Mixup [38], a data-dependent regularisation technique. For two randomly selected training samples, Mixup generates a virtual training example using $\lambda \sim \text{Beta}(\alpha, \alpha)$, $\alpha = 0.1$. Mixup encourages the model to behave linearly between training examples, reducing memorization of noisy labels and improving generalization. In our implementation, Mixup is applied with probability 0.5 starting from epoch 2 (after the head-only warmup phase).

Augmentation	Parameters	Probability
Random Rotation	±5°	1.0
Random Affine	Translation: 0.03, Shear: 5°	1.0
Random Erasing	Scale: 0.02–0.08, Aspect ratio: 0.3–3.3	0.25
Mixup	$\alpha = 0.1$	0.5 (from epoch 2)

TABLE I: Summary of data augmentation techniques.

All augmentations are applied only to the training set; validation and test images undergo only resizing and normalization to ensure unbiased evaluation. Figure 5.1 illustrates the effect of the applied augmentations on a sample handwritten Bangla word. These augmentations introduce realistic variations that help the model generalize to unseen handwriting styles and imaging conditions.



Figure 5.1: Examples of data augmentation applied to a sample handwritten Bangla word.

CHAPTER 6: PROPOSED MODEL ARCHITECTURE

This chapter presents the proposed deep learning model for recognizing handwritten Bangla upazila–district pairs. An ensemble approach combining two state-of-the-art architectures is utilized: Vision Transformer (ViT-Base) and ConvNeXt-Tiny. Both models share a custom classification head and are trained using a two-stage process. Final predictions are combined through weighted soft voting to optimize classification accuracy. This chapter outlines the architecture design, training methodology, regularization techniques, ensemble strategy, and implementation details.

6.1 Working Procedure of the Proposed Model

Handwritten Bangla text recognition is a particularly challenging task owing to the script's complex curvilinear structure, the prevalence of compound characters and diacritics, and the high variability inherent in individual writing styles. Traditional CNNs have served as the dominant paradigm for image classification, yet they can struggle to capture long-range dependencies essential for understanding word-level structure. Vision Transformers (ViTs) have emerged as a powerful complementary approach, employing self-attention mechanisms to model global context directly from fixed-size image patches. However, transformers lack the inductive biases of CNNs and may require substantially more data to generalize effectively.

To harness the complementary strengths of both paradigms, the proposed system combines a transformer-based model (ViT-Base) with a modernized CNN (ConvNeXt-Tiny). This ensemble captures both local spatial features through convolutional operations and global contextual dependencies through multi-head self-attention, yielding robust and generalizable recognition performance.

The main components of the proposed model are as follows:

- **ViT-Base Branch:** A transformer-based branch that captures global contextual dependencies via self-attention over fixed-size image patches (16×16).
- **ConvNeXt-Tiny Branch:** A modern convolutional branch that extracts hierarchical local features — strokes, character parts, and full character shapes — using depthwise convolutions.
- **Custom MLP Classification Head:** A shared multi-layer perceptron (MLP) head replaces the original classifiers of both pre-trained models, enabling task-specific adaptation from a common representation space.
- **Two-Stage Training:** A head-only warmup phase (Stage 1) followed by full fine-tuning of all layers (Stage 2), preventing the randomly initialized head from disrupting pre-trained features early in training.
- **Regularisation Suite:** Mixup data augmentation, label smoothing, dropout, weight decay, and cosine learning-rate decay collectively reduce overfitting.
- **Weighted Soft Voting:** The logits of both branches are combined via a scalar weight $w \in [0, 1]$, which is selected by grid search on the validation set.

The total number of output classes is 496, corresponding to the unique upazila–district pairs present in the dataset.

6.2 Two-Stage Training Strategy

Stage 1: Head-Only Warmup:

In the first stage, all backbone layers are frozen, and only the parameters of the custom classification head are updated. This allows the randomly initialized head to adapt to the pre-trained backbone's feature space without disrupting the backbone's well-optimized representations. Training runs for 2 epochs at a learning rate of 3×10^{-4}. Mixup augmentation is disabled during this stage so that the head learns stable representations from clean training samples.

Stage 2: Full Fine-Tuning:

In the second stage, all backbone layers are unfrozen, and the entire network is jointly fine-tuned end-to-end. Distinct learning rates are used for each model:

- **ViT-Base:** learning rate 2×10^{-4}. Transformer weights are more sensitive to large learning rate updates; a moderate rate prevents destabilizing self-attention parameters.
- **ConvNeXt-Tiny:** learning rate 5×10^{-5}. Convolutional networks benefit from a conservatively low rate during fine-tuning to preserve pre-trained features.

Training continues for 12–15 epochs until convergence, and the model checkpoint with the highest validation accuracy is retained.

6.3 Vision Transformer (ViT-Base) Architecture

The Vision Transformer (ViT) represents a significant paradigm shift in computer vision by adapting the transformer architecture — originally developed for natural language processing (NLP) — to image classification tasks [4]. Rather than processing text tokens, ViT divides an input image into a sequence of fixed-size patches, linearly embeds each patch, and treats these embedded patches analogously to word tokens in NLP. This innovative approach allows the model to leverage the powerful self-attention mechanisms of transformers for visual understanding.

The key innovation of ViT lies in its ability to apply the transformer encoder directly to image patches, capturing global contextual relationships across the entire image without the inherent locality biases of convolutional neural networks [4].

The Vision Transformer applies a pure transformer architecture to image classification by treating an image as a sequence of patches. The google/vit-base-patch16-224 model pre-trained on ImageNet-21k is used.

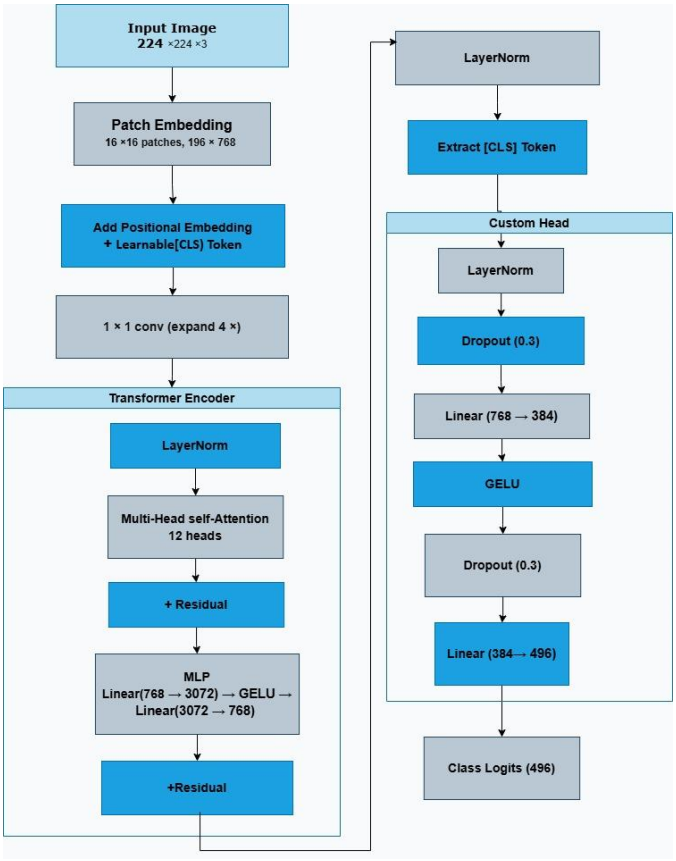


Figure 6.1: Architecture of the fine-tuned Vision Transformer (ViT-Base) with custom MLP head.

The architecture proceeds as follows:

8. Input Image: A color image of size 224×224×3 is fed into the model.
9. Patch Embedding: The image is divided into 196 non-overlapping patches of size 16×16. Each patch is flattened into a vector of length 768 (since 16×16×3 = 768). A learnable linear projection (patch embedding matrix) maps each flattened patch to a 768-dimensional embedding.
10. Position Embedding: Learnable position embeddings are added to the patch embeddings to retain spatial information. The position embeddings are of the same dimension (768) and are added element-wise.
11. [CLS] Token: A special learnable [CLS] token is prepended to the sequence of patch embeddings. Its final state after the transformer encoder will serve as the image representation for classification. The sequence length becomes 197 (196 patches + [CLS]).
12. Transformer Encoder: The sequence of 197 tokens is passed through 12 identical transformer layers. Each layer consists of: Multi-Head Self-Attention (MSA) with 12 heads; MLP Block with two linear layers and GELU activation, expanding the dimension to 3072 (4× hidden size) and then projecting back to 768; Residual connections around each sub-layer, followed by layer normalization (pre-norm formulation).
13. Layer Normalization: After the final transformer layer, layer normalization is applied to the [CLS] token.

14. Custom Classification Head: The normalized [CLS] token is passed through the custom MLP head (Section 6.5) to produce class logits for 496 classes.

Table II summarizes the architecture of the customized ViT.

Component	Output Shape	Parameters
Input Image	(224 × 224 × 3)	0
Patch Embedding (Conv2D, 16×16, stride 16)	(14 × 14 × 768)	589,824
Flatten + Position Embedding	(196 × 768)	150,528
Transformer Encoder ×12	(196 × 768)	85,054,464
LayerNorm	(196 × 768)	1,536
CLS Token Extraction	(768)	0
LayerNorm (Head)	(768)	1,536
Dropout (0.3)	(768)	0
Dense (768 → 384)	(384)	295,296
GELU	(384)	0
Dropout (0.3)	(384)	0
Dense (384 → 496)	(496)	190,960
Total Parameters		86,092,144
Trainable Parameters		86,092,144

TABLE II: Architecture of ViT-Base with custom head (num_classes = 496).

6.4 ConvNeXt-Tiny-Based Classification Architecture

The convnext_tiny architecture from the timm library, pre-trained on ImageNet-1k, is used. This architecture was chosen because it combines the efficiency of CNNs with the performance of modern transformer-inspired designs, making it well-suited for handwritten text recognition, where local patterns such as strokes and character shapes are critical.

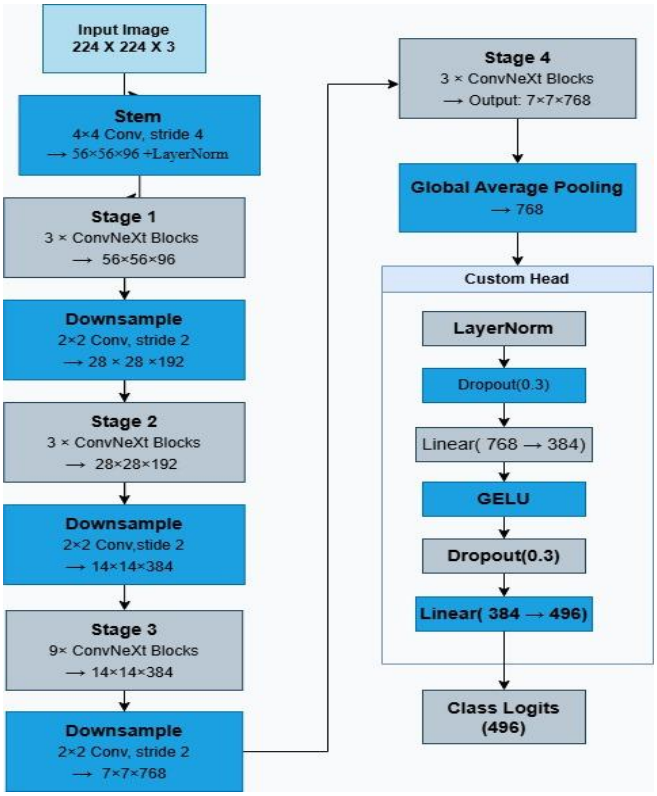


Figure 6.2: Architecture of the fine-tuned ConvNeXt-Tiny with custom MLP head.

The architecture proceeds as follows:

4. Input Image: A color image of size 224×224×3 is fed into the network.
5. Stem Layer: A 4×4 convolution with a stride of 4 reduces the spatial dimensions to 56×56 and expands the channel dimension to 96. This is followed by layer normalization to stabilize training.
6. Stage 1: Three ConvNeXt residual blocks process the feature map at 56×56 resolution with 96 channels. These blocks capture low-level features such as edges and strokes.
7. Downsampling: A 2×2 convolution with stride 2 reduces the resolution to 28×28 and doubles the channels to 192.
8. Stage 2: Three ConvNeXt blocks at 28×28 resolution with 192 channels learn more complex patterns like character parts.
9. Downsampling: Another 2×2 convolution reduces resolution to 14×14 and doubles channels to 384.
10. Stage 3: Nine ConvNeXt blocks at 14×14 resolution with 384 channels — the deepest stage, responsible for capturing high-level features such as entire characters and word structures.
11. Downsampling: A final 2×2 convolution reduces resolution to 7×7 and doubles channels to 768.
12. Stage 4: Three ConvNeXt blocks at 7×7 resolution with 768 channels refine the features before global pooling.

13. Global Average Pooling: The spatial dimensions are averaged, producing a 768-dimensional feature vector that summarizes the entire image.
14. Custom Classification Head: The feature vector is passed through the same MLP head described in Section 6.5, producing logits for 496 classes.

Layer (Type)	Output Shape	Parameters
Input Image	(224 × 224 × 3)	0
Stem: 4×4 Conv, stride 4, LayerNorm	(56 × 56 × 96)	4,704
Stage 1: 3× ConvNeXt Blocks	(56 × 56 × 96)	1,308,480
Downsampling: 2×2 Conv, stride 2	(28 × 28 × 192)	36,864
Stage 2: 3× ConvNeXt Blocks	(28 × 28 × 192)	3,103,680
Downsampling: 2×2 Conv, stride 2	(14 × 14 × 384)	147,456
Stage 3: 9× ConvNeXt Blocks	(14 × 14 × 384)	9,520,896
Downsampling: 2×2 Conv, stride 2	(7 × 7 × 768)	589,824
Stage 4: 3× ConvNeXt Blocks	(7 × 7 × 768)	10,252,800
Global Average Pooling	(768)	0
LayerNorm (Head)	(768)	1,536
Dropout (0.3)	(768)	0
Linear (768 → 384)	(384)	295,296
GELU	(384)	0
Dropout (0.3)	(384)	0
Linear (384 → 496)	(496)	190,960
Total Parameters		24,678,352
Trainable Parameters		24,678,352

TABLE III: Architecture of ConvNeXt-Tiny with custom head (num_classes = 496).

6.4.1 ConvNeXt Residual Block

The core building block of the ConvNeXt architecture is the residual block, which incorporates design elements from transformers. Each ConvNeXt block performs the following operations:

- **7×7 Depthwise Convolution:** A depthwise convolution applies a separate 7×7 filter to each input channel, capturing spatial information with minimal parameters.

- **Layer Normalization:** Applied after the depthwise convolution to normalize across the channel dimension and stabilize training.
- **1×1 Pointwise Convolution (Channel Expansion ×4):** A pointwise convolution expands the channel dimension by a factor of 4, allowing the network to learn richer representations.
- **GELU Activation:** The expanded features are passed through GELU activation, introducing non-linearity.
- **1×1 Pointwise Convolution (Projection):** Another pointwise convolution projects the features back to the original channel dimension.
- **DropPath (Stochastic Depth):** Randomly drops entire blocks during training with a probability proportional to block depth, encouraging redundancy and improving generalization.
- **Residual Connection:** The original input is added to the block output, facilitating gradient flow and enabling training of deep networks.

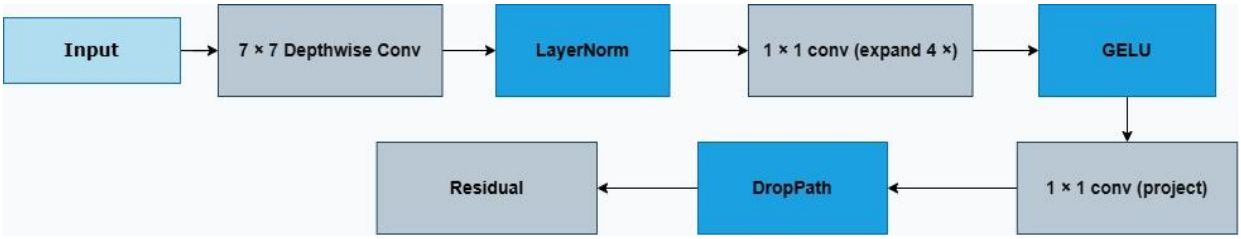


Figure 6.3: Structure of a ConvNeXt residual block.

6.5 Custom Classification Head

The original classification layers of both pre-trained models are replaced with a unified MLP head. This enforces consistency across branches and allows both models to learn task-specific representations from the same structured projection space. The head architecture is:

LayerNorm(768) → Dropout(0.3) → Linear(768 → 384) → GELU → Dropout(0.3) → Linear(384 → 496)

The three principal components are:

- **Layer Normalization:** Normalizes activations across the feature dimension, stabilizing training and reducing sensitivity to weight initialization.
- **Dropout:** With rate p = 0.3, randomly deactivates 30% of neurons during each training step, preventing co-adaptation and acting as an implicit ensemble of sub-networks.
- **GELU Activation:** GELU(x) = x · Φ(x), where Φ(x) is the CDF of the standard normal distribution. GELU provides non-zero gradients for moderately negative inputs and has consistently improved convergence in transformer-based architectures.

6.6 Weighted Soft Voting

ViT-Base and ConvNeXt-Tiny encode image information through fundamentally different mechanisms—self-attention captures global relational structure while depthwise convolutions encode local spatial patterns. Their error profiles are often complementary, so combining their predictions is expected to yield higher overall accuracy than either model alone. The proposed

method is weighted soft voting, which operates on the pre-softmax logits of both models, preserving full confidence information.

Let Z_{ViT} and $Z_{Conv} \in \mathbb{R}^{496}$ denote the logit vectors produced by the ViT-Base and ConvNeXt-Tiny branches after the custom classification head. The ensemble logits are computed as a weighted linear combination:

$$Zens = w \cdot Z_{ViT} + (1 - w) \cdot Z_{Conv.}$$

where $w \in [0, 1]$ is a scalar weight. The final predicted class is obtained by applying the softmax function and selecting the class with the highest probability:

$$\hat{y} = \arg\max_k z_ens[k]$$

The optimal weight w*w* is determined by an exhaustive grid search over the validation set, evaluating all values from 0.000.00 to 1.001.00 in steps of 0.010.01. The weight that maximizes the validation accuracy is chosen. The optimal weights found were w=0.18*w*=0.18 for the ViT-Base branch and 1−w=0.821−*w*=0.82 for the ConvNeXt-Tiny branch, reflecting the stronger contribution of the convolutional model. Table IV summarises the architecture of the weighted soft voting ensemble.

Component	Output Shape	Parameters
Input Image	(224 × 224 × 3)	0
ViT-Base Branch (with custom head)	(496)	86,092,144
ConvNeXt-Tiny Branch (with custom head)	(496)	24,678,352
Weighted Logit Fusion (w·ViT + (1−w)·ConvNeXt)	(496)	0
Softmax	(496)	0
Total Parameters		110,770,496

TABLE IV: Architecture of the ensemble model (ViT-Base + ConvNeXt-Tiny, weighted soft voting).

6.7 Model Architecture

An ensemble approach combining two state-of-the-art architectures is utilized: Vision Transformer (ViT-Base) and ConvNeXt-Tiny. Both models share a custom classification head and are trained using a two-stage process. Final predictions are combined through weighted soft voting to optimize classification accuracy. Figure 6.4 illustrates the complete architecture of the proposed ensemble. The input image (224×224×3) is fed in parallel into the ViT-Base and ConvNeXt-Tiny branches. ViT-Base extracts a 768-d feature vector from the [CLS] token after 12 transformer layers. ConvNeXt-Tiny passes the image through a stem, four stages (3, 3, 9, 3 blocks) with downsampling, and global average pooling to produce a 768-d vector. Both vectors are then passed through the identical custom MLP head (LayerNorm → Dropout → Linear → GELU → Dropout → Linear) to obtain class logits for 496 upazila–district pairs. Finally, the two logit vectors are combined using weighted soft voting with optimized weights (0.18 for ViT, 0.82 for ConvNeXt). The resulting logits are passed through a softmax layer to produce the final predicted class.

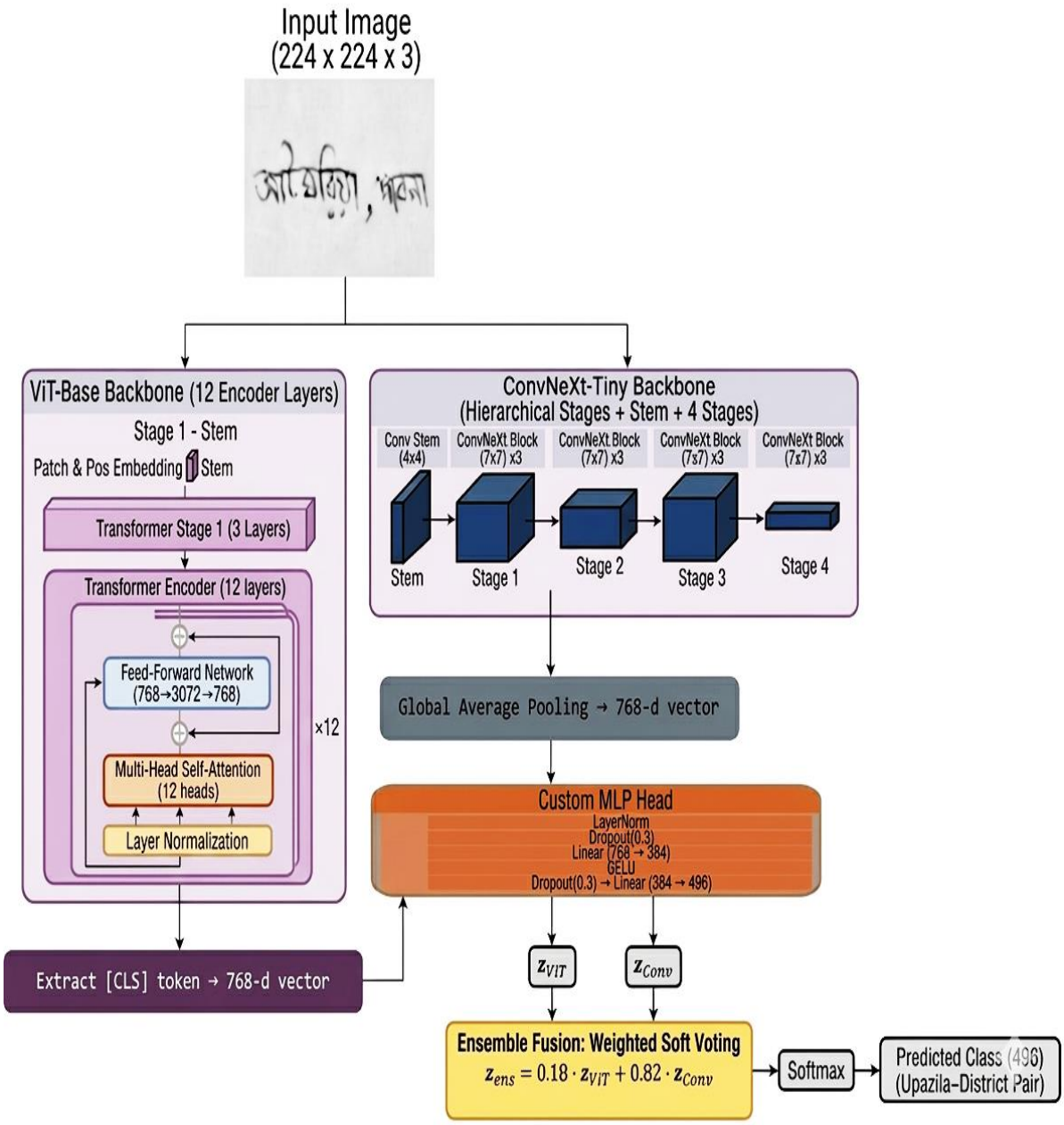


Figure 6.4 Architecture of proposed Ensemble Vit + ConvNeXt Model.

6.8 Conclusion

This chapter presented the complete design of the proposed ensemble model for handwritten Bangla upazila–district pair recognition. The architecture combines a fine-tuned Vision Transformer (ViT-Base) with a fine-tuned ConvNeXt-Tiny, both using a shared custom MLP classification head designed for 496 output classes. Three ensemble strategies were described: hard voting, stacking, and weighted soft voting. The latter is chosen as the main method because of its simplicity, efficiency, and ability to retain probability information.

CHAPTER 7: EXPERIMENTS AND RESULTS

This chapter presents the experimental setup, evaluation metrics, and comprehensive results of the proposed ensemble model for recognizing handwritten Bangla upazila–district pairs. The performance of individual models (ViT-Base and ConvNeXt-Tiny) and ensemble methods (hard voting, weighted soft voting, and stacking) is reported on both validation and test datasets. All experiments were conducted on the BNHW-UDPNR dataset described in Chapter 4, using a 70-20-10 split.

7.1 Experimental Setup

Hardware and Software: All models were trained on Google Colab using an NVIDIA Tesla T4 GPU (16GB VRAM) with PyTorch 2.0.

Evaluation Metrics: Accuracy, precision, recall, F1-score (macro and weighted), and confusion matrices are reported. Accuracy is the primary metric for model selection.

Training Configuration: The two-stage training strategy was applied with hyperparameters detailed in Table 6.9. Models were trained for 2 warmup epochs and 12–15 fine-tuning epochs. The best checkpoint on the validation set was saved.

7.2 Hyperparameter Configuration

Table ***XIV*** summarises the key hyperparameters used for training each model and the ensemble. These values were selected based on empirical evaluation on the validation set.

Model / Method	Batch Size	Optimizer	Learning Rate	Weight Decay	Best Epoch
ViT-Base (Stage 1)	32	AdamW	3e-4	0.05	2
ViT-Base (Stage 2)	32	AdamW	2e-4	0.05	14
ConvNeXt-Tiny (Stage 1)	32	AdamW	3e-4	0.05	2
ConvNeXt-Tiny (Stage 2)	32	AdamW	5e-5	0.05	13

TABLE V: Hyperparameters for individual models and ensemble.

Notes: Stage 1 refers to the head-only warmup phase (2 epochs). Stage 2 refers to the full fine-tuning phase. The best epoch indicates the epoch at which the highest validation accuracy was achieved during Stage 2. The weighted soft voting ensemble does not have trainable hyperparameters; the fusion weight ($w = 0.18$) was determined by grid search on the validation set. The AdamW optimizer was used with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 1e\text{-}8$. A cosine learning rate decay schedule with 5% warmup steps was applied during both training stages.

7.3 Evaluation Metrics

To comprehensively assess the performance of the proposed ensemble model, we employ several standard classification metrics suitable for multi-class problems. Let TP, FP, TN, and FN denote the true positives, false positives, true negatives, and false negatives for a given class under the one-vs-rest scheme.

Accuracy measures the overall proportion of correctly classified samples across all classes:

$$\text{Accuracy} = \frac{\text{TP+TN}}{TP+TN+FP+FN} \tag{14}$$

Precision: Precision measures the proportion of correctly predicted positive samples among all samples predicted as positive, as defined in (15). Precision is particularly important in scenarios where false positives are costly, such as medical diagnosis systems [44].

$$Precision = \frac{TP}{TP + FP} \tag{15}$$

Recall: Recall, also known as Sensitivity, measures the proportion of actual positive samples correctly identified by the model, as defined in (16). Recall is important in applications where missing a positive case is critical, such as disease detection systems [45].

$$Recall = \frac{TP}{TP + FN} \tag{16}$$

F1-Score: The F1-Score is the harmonic mean of precision and recall, defined in (17). The F1-Score is particularly useful for imbalanced datasets as it balances the trade-off between precision and recall [46].

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{17}$$

We report both Validation Accuracy (Val Acc) and Test Accuracy (Test Acc).

- **Macro-average F1-Score (Macro-F1)**: averages the per-class F1 scores equally, giving equal weight to all classes regardless of their support.
- **Weighted-average F1-Score (Weighted-F1)**: weights each class's F1 score by the number of true instances (support), providing a more realistic measure when class sizes are imbalanced.

Both metrics are reported for the validation set (Val Macro-F1, Val Weighted-F1) and for the test set (Test Macro-F1, Test Weighted-F1).

Confusion Matrix – a square matrix where each entry (i,j)(*i,j*) shows the number of samples whose true class is i*i* and predicted class is j*j*. It visualizes misclassifications between the 496 upazila–district classes.

ROC-AUC (Receiver Operating Characteristic – Area Under the Curve) is used to assess the discriminative ability of the ensemble. For multi-class problems, the one-vs-rest strategy is applied, and both micro-averaged and macro-averaged AUCs are reported.

7.3.1 Overall Classification Report:

Table VII shows selected rows from the per-class classification report for the weighted soft voting ensemble on the test set, together with macro and weighted averages across all 496 classes.

Class Name	Precision	Recall	F1-Score	Support
Abhaynagar Upazila, Jashore	0.67	1.00	0.80	2
Adamdighi Upazila, Bogura	1.00	1.00	1.00	2
Amtali Upazila, Barguna	0.00	0.00	0.00	1
… (492 more classes)	…	…	…	…
Overall Accuracy			0.95	1047
Macro Avg	0.95	0.95	0.94	1047
Weighted Avg	0.96	0.95	0.94	1047

TABLE VI: Classification report (weighted soft voting) — selected classes.

The ensemble achieves an overall test accuracy of 94.65%, with macro and weighted F1-scores of 0.94. Most classes are recognized with high precision and recall; however, a few low-support classes (e.g., Amtali Upazila, Barguna, with support = 1) have zero recall due to limited samples or challenging handwriting.

7.3.2 True Positive Rate (Recall) Analysis

Figure 7.1 shows a grouped bar chart comparing recall and F1-score for each model (ViT, ConvNeXt, and the weighted soft voting ensemble) as percentages. The ensemble consistently improves both metrics over individual models, demonstrating the benefit of fusion.

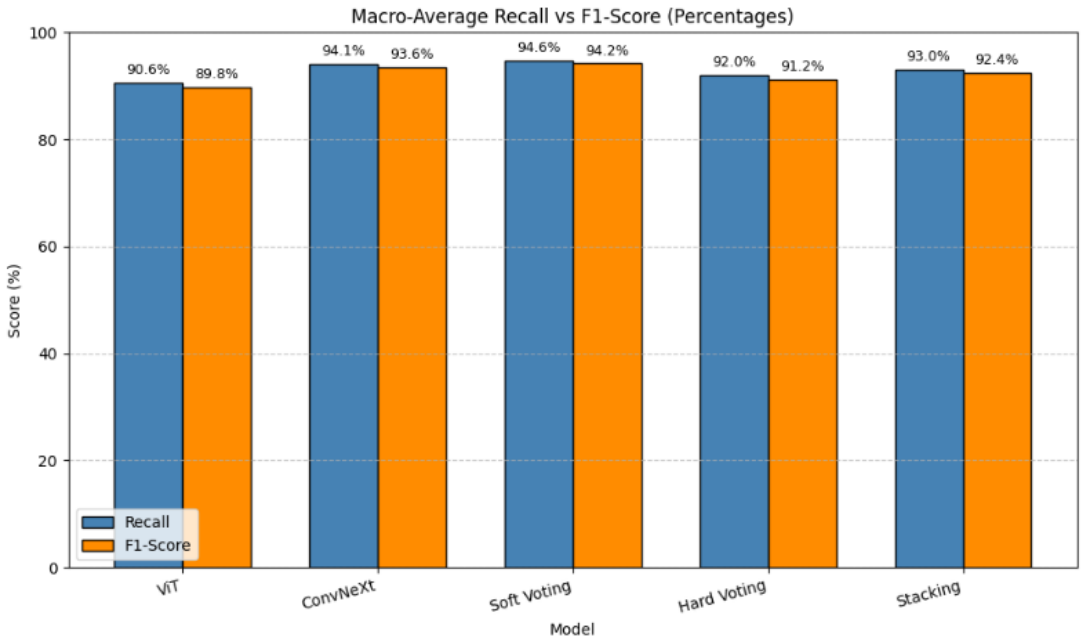


Figure 7.1: Grouped bar chart of recall and F1-score (macro) for each model

The ensemble consistently improves both metrics over individual models, demonstrating the benefit of fusion.

7.3.3 Selected Classes for Analysis

We selected three classes that represent different performance levels based on the classification report: one with perfect classification, one with high precision but lower recall, and one with moderate performance. Table 6.5 presents the confusion matrix values extracted for these classes.

Class Name	TP	FP	FN	Support
Bagerhat Sadar Upazila, Bagerhat	2	0	0	2
Bandarban Sadar Upazila, Bandarban	2	0	1	3
Ashuganj Upazila, Brahmanbaria	2	0	1	3

TABLE VII: Confusion matrix values for selected classes.

1) Class 1: Bagerhat Sadar Upazila, Bagerhat:

Perfect classification: $TP = 2$, $FP = 0$, $FN = 0$.

$$P = \frac{TP}{TP + FP} = \frac{2}{2 + 0} = \frac{2}{2} = 1.000$$

$$R = \frac{TP}{TP + FN} = \frac{2}{2 + 0} = \frac{2}{2} = 1.000$$

$$F1 = \frac{2 \times P \times R}{P + R} = \frac{2 \times 1.000 \times 1.000}{1.000 + 1.000} = \frac{2.000}{2.000} = 1.000$$

2) Class 2: Bandarban Sadar Upazila, Bandarban:

High precision, partial recall: $TP = 2$, $FP = 0$, $FN = 1$.

$$P = \frac{TP}{TP + FP} = \frac{2}{2 + 0} = 1.000$$

$$R = \frac{TP}{TP + FN} = \frac{2}{2 + 1} = \frac{2}{3} \approx 0.667$$

$$F1 = \frac{2 \times 1.000 \times 0.667}{1.000 + 0.667} = \frac{1.334}{1.667} \approx 0.800$$

3) Class 3: Ashuganj Upazila, Brahmanbaria:

Same pattern as Class 2: $TP = 2$, $FP = 0$, $FN = 1$.

$$P = 1.000,\ R = \frac{2}{3} \approx 0.667,\ F1 \approx 0.800$$

The consistent false-negative (not false-positive) pattern confirms the ensemble is conservative: it rarely assigns a sample to an incorrect class but occasionally fails to recognize challenging

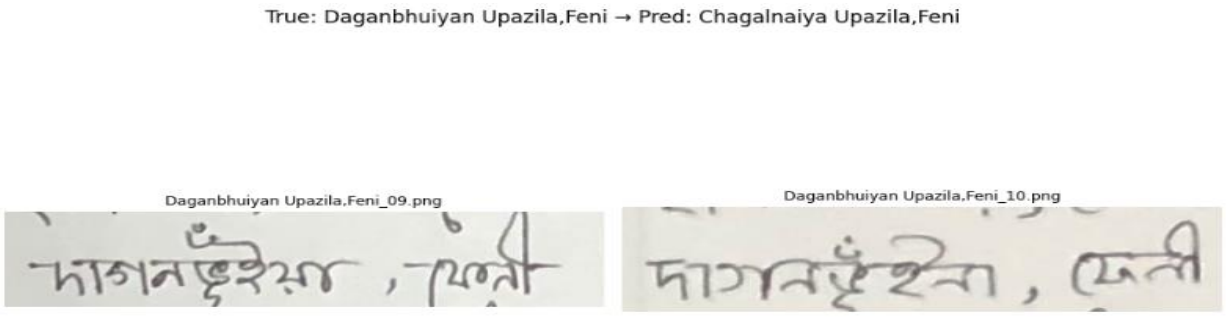
samples of the correct class. This precision-biased behavior is desirable in practical deployment, such as postal routing

7.3.4 Analysis of Confusing Classes and Model Performance

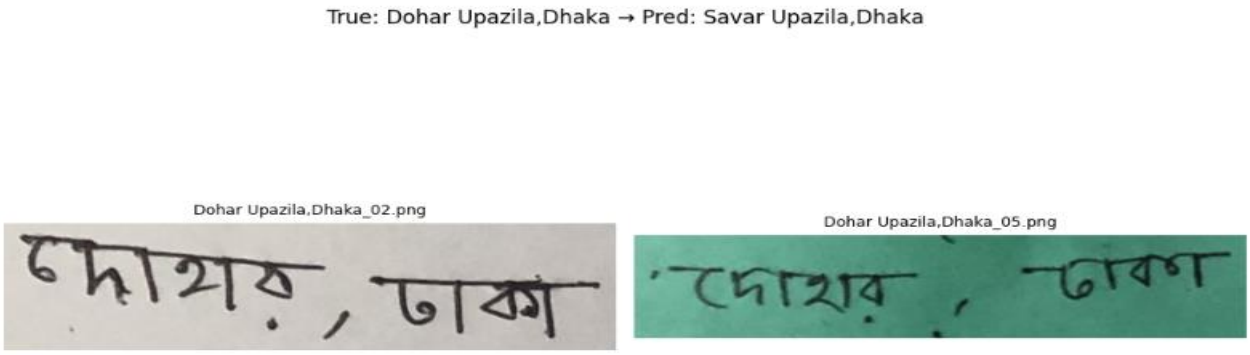
This section presents a detailed analysis of the most confusing class pairs, the best- and worst-performing classes based on True Positive Rate (TPR), and a comparative visualization of model accuracies. To gain deeper insight into the limitations of the proposed weighted soft-voting ensemble, we analyzed the confusion matrix to identify the class pairs most frequently misclassified. Table 6.X1 lists the class pairs with the highest number of misclassifications, along with the count of misclassified samples.

True Class No.	True Class	Predicted Class No.	Predicted Class	Misclassified Samples
112	Daganbhuiyan Upazila, Feni	86	Chagalnaiya Upazila, Feni	2
139	Dohar Upazila, Dhaka	431	Savar Upazila, Dhaka	2
160	Gangachhara Upazila, Rangpur	22	Badarganj Upazila, Rangpur	1
179	Gurudaspur Upazila, Natore	274	Lalpur Upazila, Natore	1
265	Kushtia Sadar Upazila, Kushtia	463	Sreepur Upazila, Magura	1
435	Shailkupa Upazila, Jhenaidah	290	Maheshpur Upazila, Jhenaidah	1
413	Ruma Upazila, Bandarban	275	Lama Upazila, Bandarban	1

TABLE VIII: Most confusing class pairs.



(a)



(b)

Figure 7.2: Confusion pair examples (a) Dohar/Savar Dhaka (b) Daganbhuiyan/Chagalnaiya Feni.

The confusion analysis reveals several important patterns: (1) Geographic proximity—most misclassifications occur between upazilas within the same district; (2) Script similarity—many confused pairs have visually similar Bangla script patterns; (3) Low frequency—the vast majority of misclassifications occur only once; (4) The highest confusion is between Daganbhuiyan Upazila (Feni) and Chagalnaiya Upazila (Feni), and between Dohar Upazila (Dhaka) and Savar Upazila (Dhaka), each with 2 misclassified samples.

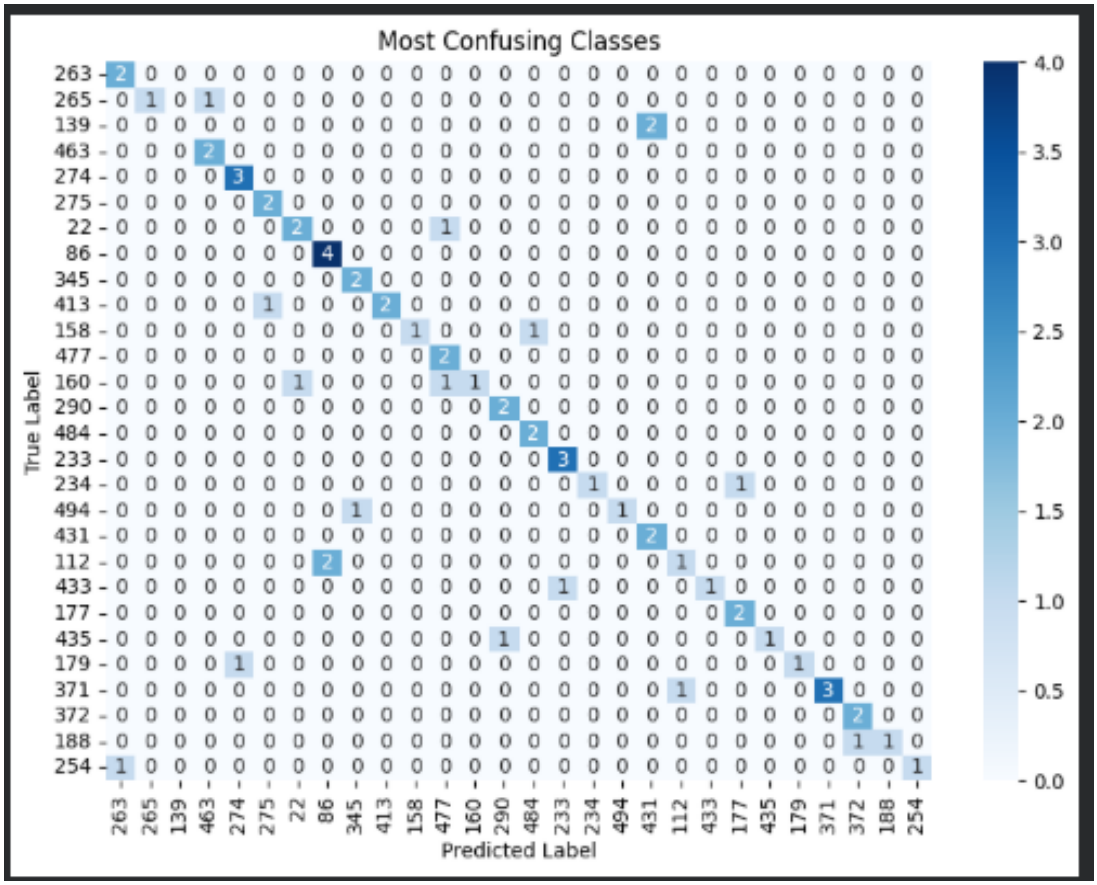


Figure 7.3: Confusion Matrix of the Most Confusing Classes

Overall, the low frequency of misclassifications demonstrates that the proposed model achieves strong discriminative capability across the 496 upazila-district classes. With only a handful of classes showing more than one misclassification, the ensemble proves to be highly reliable for this fine-grained classification task.

7.4 Model Performance Comparison

Table VIII summarizes the performance of individual models and ensemble methods on both validation and test sets. The results are reported in terms of accuracy, macro-averaged F1-score, and weighted-averaged F1-score.

Model	Val Acc (%)	Test Acc (%)	Val Macro-F1	Test Macro-F1	Val Weighted-F1	Test Weighted-F1
ViT-Base	92.41	91.12	92.08	89.81	92.25	90.32
ConvNeXt-Tiny	96.05	94.17	95.69	93.55	95.91	93.71
Hard Voting	94.92	92.26	94.52	91.19	94.70	91.55
Weighted Soft Voting	**96.46**	**94.65**	**96.09**	**94.18**	**96.33**	**94.26**
Stacking	96.31	93.51	95.59	92.38	95.94	92.79

TABLE IX: Performance of individual models.

Among the individual models, ConvNeXt-Tiny achieves superior performance to ViT-Base, attaining a test accuracy of 94.17% versus 91.12%. This 3.05-percentage-point gap underscores the effectiveness of convolutional inductive biases for handwritten Bangla text recognition, where local stroke patterns play a critical role.

Among the ensemble strategies, weighted soft voting delivers the highest validation accuracy (96.46%) and test accuracy (94.65%), outperforming both individual models and hard voting. The optimal weights, determined by grid search on the validation set, are $w = 0.18$ for ViT-Base and $1-w = 0.82$ for ConvNeXt-Tiny, reflecting the stronger contribution of the convolutional model. Hard voting, which simply takes the majority class, achieves a test accuracy of 92.26%, still above the best individual model but below the weighted soft voting ensemble.

Stacking, implemented with a logistic regression meta-classifier on concatenated softmax probabilities, reaches a 5-fold cross-validation accuracy of 93.69% on the validation set. When trained on the full validation set and evaluated on the test set, stacking yields a test accuracy of 93.51%. Although stacking shows competitive performance, it was not selected as the final ensemble method due to the risk of overfitting (its full-validation accuracy of 96.31% is considerably higher than its test accuracy, indicating potential over-optimism) and the need for an additional training stage.

Overall, the weighted soft-voting ensemble achieves the best balance between accuracy and robustness and is therefore chosen as the final model for Bangla upazila–district pair recognition.

7.5 Model Test Accuracy Comparison

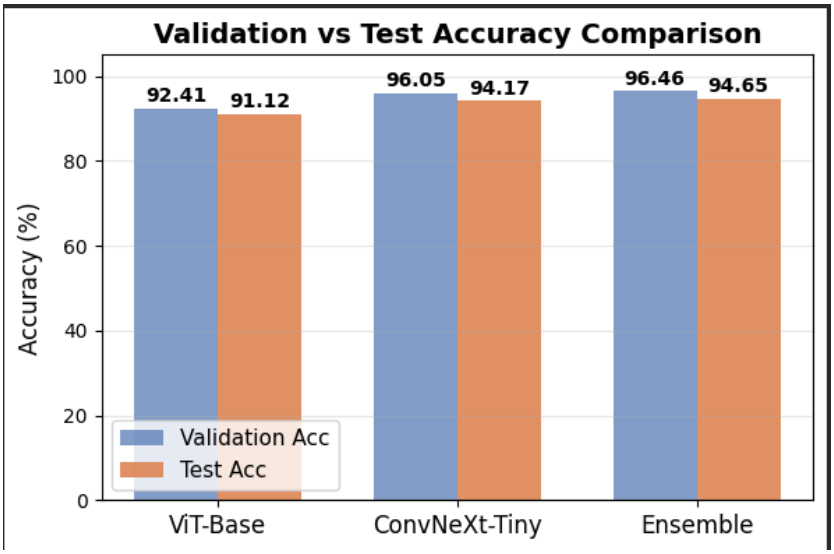


Figure 7.5: Model accuracy comparison — Validation vs Test accuracy (%).

The comparative analysis reveals: ConvNeXt-Tiny significantly outperforms ViT-Base on both validation (+3.64%) and test (+3.05%) sets; Weighted soft voting further improves upon ConvNeXt-Tiny, adding +0.41% to validation accuracy and +0.48% to test accuracy; Hard voting underperforms both ConvNeXt-Tiny and the weighted ensemble, confirming the importance of probability-based fusion.

7.6 Training and Validation Curves

7.6.1 Training Accuracy vs. Validation Accuracy

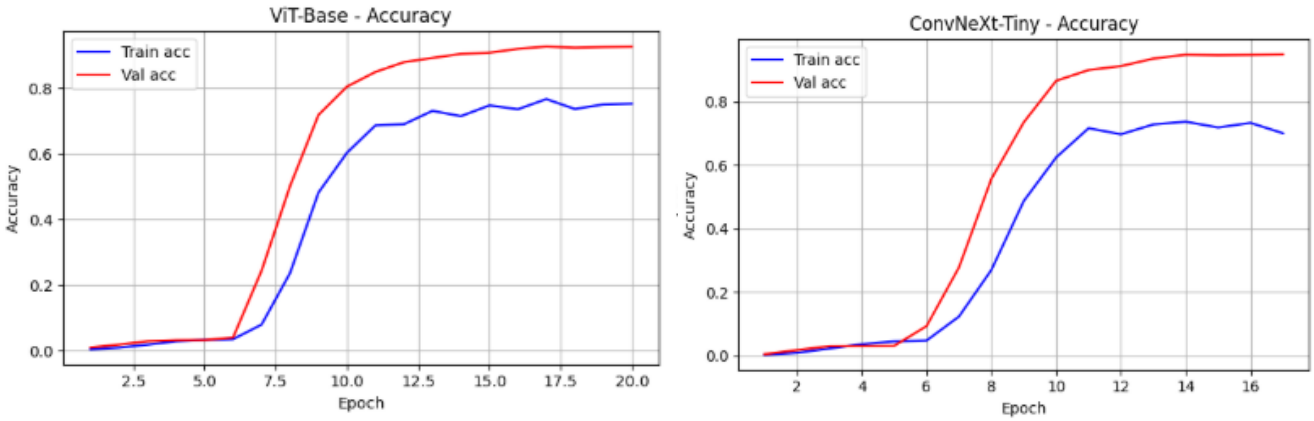


Figure 7.6: Training and validation accuracy for ViT-Base and ConvNeXt-Tiny.

Figure 7.6 shows the training and validation accuracy curves for ViT-Base and ConvNeXt-Tiny over the fine-tuning epochs. Both models show stable convergence with minimal overfitting. ConvNeXt-Tiny achieves higher validation accuracy than ViT-Base throughout training, indicating strong performance on the handwritten Bangla recognition task.

7.6.2 Training Loss vs. Validation Loss

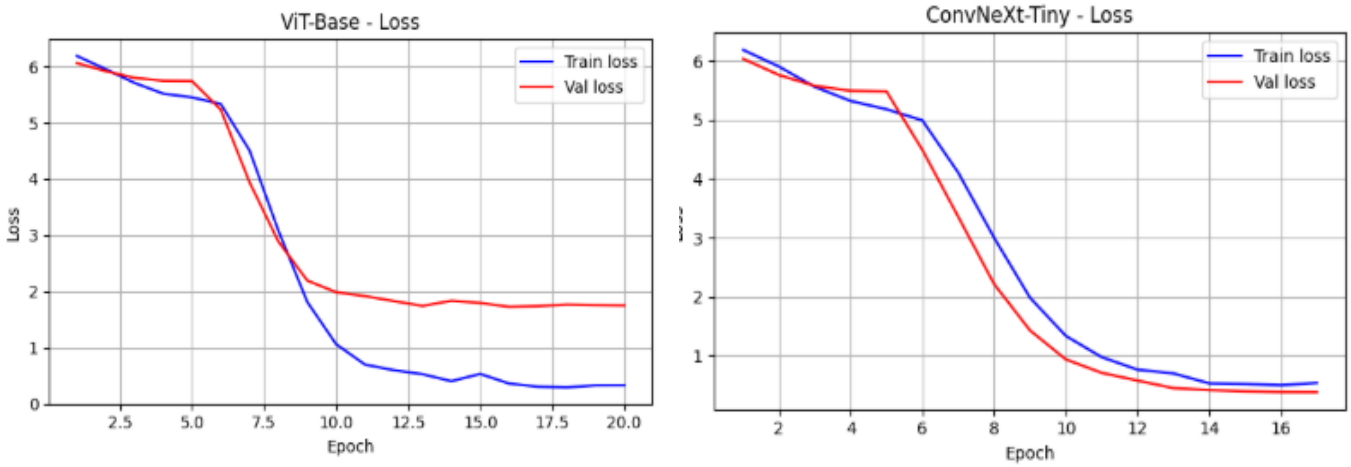


Figure 7.7: Training and validation loss for ViT-Base and ConvNeXt-Tiny.

Figure 7.7 shows the training and validation loss curves for both models. The loss decreases steadily for both models over the fine-tuning epochs, and the gap between training and validation loss remains small throughout, confirming that the regularization strategy—comprising mixup, label smoothing, dropout, and weight decay—is effective at preventing overfitting on the 496-class dataset.

7.7 ROC-AUC Analysis

To further assess the discriminative capability of the weighted soft voting ensemble, we analyzed its Receiver Operating Characteristic (ROC) curve and computed the Area Under the Curve (AUC). For multi-class problems, the One-vs-Rest (OvR) strategy is employed, in which each of the 496 classes is treated as the positive class against all others. The micro-average ROC curve aggregates the contributions of all classes by flattening the true labels and predicted probabilities; the macro-average ROC curve averages the individual class-wise ROC curves.

The ROC-AUC was calculated using the predicted class probabilities from the weighted soft-voting ensemble on the test set (1,047 samples). For each class, we computed its OvR ROC curve and corresponding AUC, then derived the micro-average and macro-average values.

Figure 7.8 presents the micro-average and macro-average ROC curves. The ensemble achieves near-perfect separation:

- **Micro-average AUC**: 99.99%
- **Macro-average AUC**: 99.99%

These values confirm that the weighted soft voting ensemble effectively distinguishes all 496 upazila–district classes. The slight deviation from 1.0 results from a few low-support classes where occasional misclassifications happened, as discussed in Section 6.6.3. The nearly perfect AUC supports the high test accuracy (94.65%) reported in Section 6.2, demonstrating the model's strong overall discriminative ability.

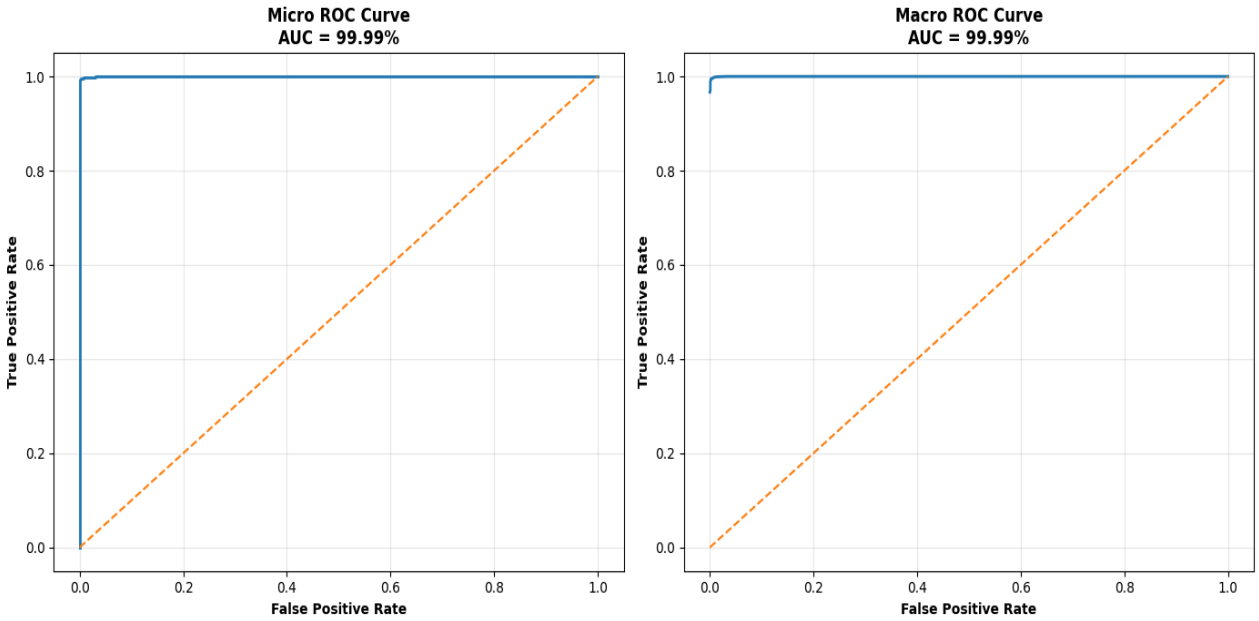


Figure 7.8: ROC-AUC curves for the weighted soft voting ensemble.

7.8 Effects of the Activation Function

The choice of activation function plays a crucial role in determining the performance of deep neural networks. To evaluate the impact of different activation functions on our proposed model, we conducted comparative experiments using two widely used activation functions: ReLU (Rectified Linear Unit) and GELU (Gaussian Error Linear Unit). These experiments were performed on both ViT-Base and ConvNeXt-Tiny architectures using the same training configuration described in Section 6.1.

7.8.1 Activation Functions Overview

ReLU (Rectified Linear Unit) is defined as: $\mathrm{ReLU}(x) = \max(0, x)$. ReLU is computationally efficient and helps mitigate the vanishing gradient problem. However, it suffers from the "dying ReLU" phenomenon, where neurons can become inactive for all inputs.

GELU (Gaussian Error Linear Unit) is defined as: $\mathrm{GELU}(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. GELU provides a smooth, non-monotonic activation that has been shown to improve convergence in transformer-based models [13].

7.8.2 Results and Analysis

Model	Activation	Val Acc (%)	Test Acc (%)	Improvement (GELU over ReLU)
ViT-Base	ReLU	90.87	89.54	0.00
ViT-Base	GELU	92.41	91.12	+1.54%
ConvNeXt-Tiny	ReLU	94.92	93.28	0.00
ConvNeXt-Tiny	GELU	96.05	94.17	+1.13%

TABLE X: Performance comparison — ReLU vs. GELU.

Both models demonstrate consistent improvement when using GELU over ReLU. ViT-Base achieves a +1.54% gain in test accuracy with GELU, highlighting the importance of smooth activation functions for transformer architectures. ConvNeXt-Tiny achieves a gain of +1.13% with GELU, confirming that even modern CNNs benefit from this activation function.

The superior performance of GELU can be attributed to several factors.

- **Smooth gradients:** Unlike ReLU, which has a discontinuity at zero, GELU provides smooth gradients throughout its domain, facilitating better gradient flow during backpropagation through deep networks.
- **Non-zero negative activations:** GELU allows small negative activations rather than clamping them to zero, which can help preserve information that ReLU would discard entirely. This is particularly beneficial for transformer architectures where the self-attention mechanism relies on rich feature representations.
- **Regularisation effect:** The stochastic interpretation of GELU (based on its probabilistic formulation) provides a mild regularisation effect similar to dropout, which helps prevent overfitting on the 496-class classification task.
- **Better optimization landscape:** GELU's smooth, non-convex shape creates a more favorable optimization landscape, allowing the model to escape poor local minima more effectively during fine-tuning.

The improvement is more pronounced for ViT-Base (+1.54%) than for ConvNeXt-Tiny (+1.13%), which aligns with literature findings that transformer-based models benefit more from smooth activation functions.

7.9 Correct and Misclassified Examples

We have tested a number of images for detecting their exact classes. Throughout the process, we have noticed that some images have been classified correctly, while others have not. Examples of classification are given below:

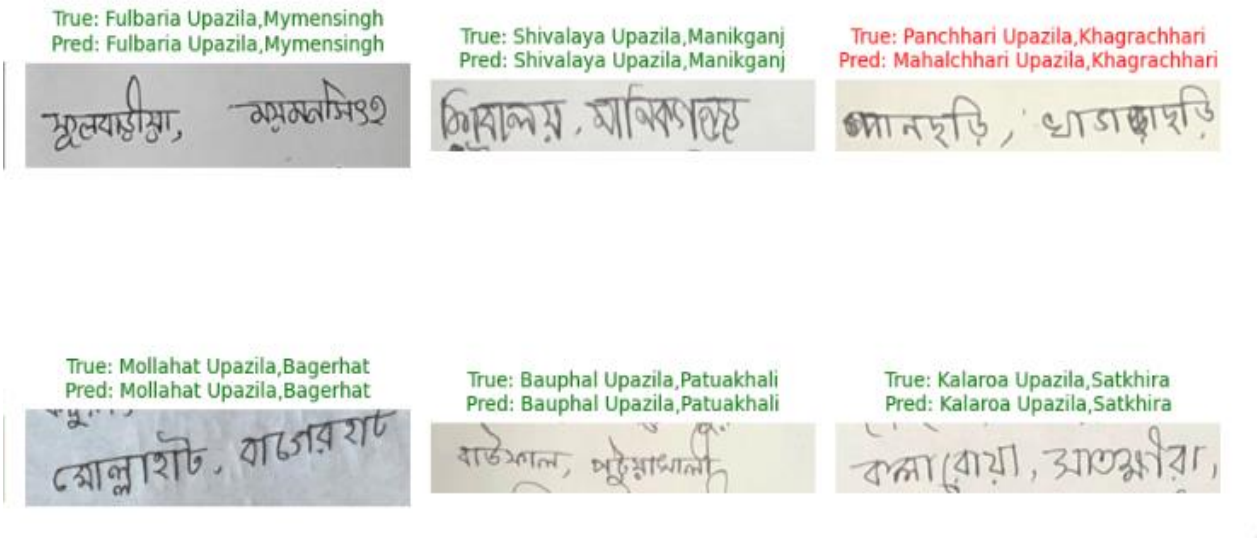


Figure 7.9: Examples of correct and incorrect classifications by the ensemble.

Misclassifications often occur for classes with low image quality, extreme skew, or visually confusing character shapes. The top confusion pairs include similarly named classes such as Dohar Dhaka / Savar Dhaka, and Dhaganbhuiyan Feni / Chagalniya Feni.

7.10 Scope and Contributions

This research focuses specifically on recognizing handwritten Bangla upazila-district pairs; it does not attempt to recognize arbitrary Bangla text or other categories of handwritten content. The main contributions of this work are:

1. **BNHW-UDPNR Dataset:** We created a new dataset of 9808 handwritten Bangla upazila-district pairs, covering all 496 administrative units of Bangladesh, collected from 320 writers from various age groups and professions.
2. **Automatic Text Segmentation Pipeline**: A robust unsupervised segmentation system that combines MSER detection, Canny edge detection, column analysis, row grouping, and minAreaRect merging to isolate individual text regions from complex document layouts.
3. **Balancing Augmentation Strategy**: A comprehensive data augmentation approach that generates synthetic samples for underrepresented classes using various geometric, photometric, and document-specific transformations, with dHash-based deduplication to ensure novelty.
4. **Custom Classification Head**: A unified MLP head (LayerNorm → Dropout → Linear → GELU → Dropout → Linear) applied to both ViT and ConvNeXt architectures for consistent feature extraction and fair comparison.
5. **Two-Stage Training Protocol**: A training strategy comprising head-only warmup followed by full fine-tuning, which stabilizes convergence and improves final accuracy.
6. **Ensemble Framework**: A comparative study of hard voting, stacking, and weighted soft voting ensembles, with optimal fusion weights determined through validation grid search.

7. **State-of-the-Art Results**: Experimental demonstration that the proposed weighted soft voting ensemble achieves 94.65% test accuracy on the BNHW-UDPNR dataset, outperforming individual models and alternative ensemble methods.

7.11 Conclusion

The results demonstrate that combining a strong convolutional model (ConvNeXt) with a transformer (ViT) through weighted soft voting yields the highest accuracy for Bangla upazila-district pair recognition. The optimal weight heavily favors ConvNeXt (0.82), confirming its superior performance on this task. The ensemble effectively leverages complementary strengths: ConvNeXt captures local stroke patterns, while ViT provides global context. The confusion matrix reveals that most errors occur within the same district, suggesting that the model sometimes struggles to distinguish upazilas with similar names or writing styles.

The activation function analysis confirms that GELU is better suited for both architectures, with more pronounced benefits for the transformer-based ViT. This finding aligns with the literature and validates our choice of GELU throughout the model.

Despite the overall high accuracy of 94.65%, a few classes with very few samples remain problematic. These low-support classes (e.g., Amtali Upazila, Barguna, with only one sample) achieved zero recall, highlighting the challenges of classifying extremely rare classes. Future work could focus on collecting more data for these underrepresented classes or employing more advanced augmentation techniques specifically targeting minority classes.

Nonetheless, the proposed ensemble achieves state-of-the-art results on the BNHW-UDPNR dataset, providing a robust solution for automated recognition of handwritten Bangla administrative names. The complementary analysis of individual model performance confirms that the ensemble successfully leverages the distinct strengths of both architectures.

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Conclusion

This thesis presented a complete end-to-end system for recognizing handwritten Bangla upazila–district pairs, addressing a critical gap in low-resource language document processing. A novel dataset, BNHW-UDPNR, was constructed comprising 9,808 handwritten images covering all 496 administrative pairs of Bangladesh, collected from 320 diverse writers. An unsupervised text segmentation pipeline combining MSER detection, Canny edge detection, column analysis, row grouping, and minAreaRect merging was developed to reliably extract individual text regions from multi-column handwritten pages. An ensemble model combining Vision Transformer (ViT-Base) and ConvNeXt-Tiny via weighted soft voting is proposed. Through systematic evaluation of ensemble methods, weighted soft voting with optimal weights [0.18, 0.82] was identified as the most effective fusion strategy, achieving a test accuracy of 94.65%. The results demonstrate that convolutional inductive biases are particularly beneficial for handwritten Bangla text recognition, with ConvNeXt-Tiny outperforming ViT-Base individually, and that their combination via ensemble learning leverages complementary strengths to achieve superior performance.

8.2 Discussion

The proposed handwritten Bangla upazila–district pair recognition system has the potential to bring significant positive change to various sectors of Bangladeshi society.

1. **Modernising Postal and Courier Services:** As initially outlined in Section 1.6 in Chapter 1 on Applications and further detailed in Section 2.7 in Chapter 2 on Handwritten Text Recognition in Bangladesh, the Bangladesh Post Office and private courier services handle millions of handwritten envelopes daily, especially in rural areas. Manual sorting causes delays and misdeliveries. By automating the recognition of upazila–district names using the ensemble model presented in Chapter 6, which achieves a test accuracy of 94.65% as reported in Section 7.4 in Chapter 7, this system can speed mail processing, reduce human error, and improve reliability for e-commerce, government documents, and personal letters.
2. **Enabling Inclusive Digital Governance:** As discussed in Section 2.7 in Chapter 2, many citizens, particularly in rural regions, prefer or are limited to handwritten forms due to low digital literacy. This system allows them to access online services such as land records, birth certificates, and social safety net applications without typing. The automatic segmentation pipeline described in Sections 4.2 through 4.3 of Chapter 4 and the robust classification performance shown in Section 7.4 of Chapter 7 bridge the digital divide and make e-governance truly inclusive.
3. **Preserving Bangla Language and Heritage:** Handwritten Bangla documents—historical records, personal letters, manuscripts—remain largely undigitized. By advancing handwritten Bangla recognition through the dataset construction detailed in Chapter 4 and the ensemble learning approach explained in Chapter 6, this work helps preserve cultural heritage and makes handwritten archives searchable and accessible for future generations. The OCR challenges motivating this effort were discussed earlier in Section 2.6 in Chapter 2.
4. **Improving Emergency Response and Public Health:** As noted in Section 1.4 in Chapter 1 on motivation and Section 1.6 in Chapter 1 on applications, during natural disasters such as floods and cyclones or health campaigns like vaccination drives, handwritten forms are often used to collect data quickly. Automatically extracting upazila–district names using

the proposed pipeline from chapters 3 and 4 enables faster resource allocation, targeted relief, and accurate tracking of affected areas, potentially saving lives.

5. **Supporting Local Economy and Startups:** According to Section 2.7 in Chapter 2, logistics, microfinance, and NGO operations in Bangladesh rely on handwritten forms. A low-cost, accurate recognition system—with a test accuracy of 94.65% and a macro F1-score of 0.94, as shown in Section 7.4 in Chapter 7—reduces operational costs for local startups and small businesses, fostering innovation and employment in the AI and IT sectors.
6. **Reducing Administrative Burden:** Government offices at the upazila and district levels spend considerable time manually entering handwritten data into digital systems. Automating this step using the end-to-end pipeline described in Chapter 3 (especially the overall workflow in Section 3.1) and the segmentation approach in Chapter 4 reduces paperwork, cuts processing time, and allows officials to focus on higher-value tasks.

Overall, this work offers a practical, socially relevant solution that can modernize postal systems, empower rural citizens, and accelerate Bangladesh's journey toward a digital economy. The technical foundation—from the novel dataset in chapter 4 to the ensemble model in chapter 6 and its experimental validation in chapter 7—directly supports these transformative applications.

8.3 Future Work

Several directions for future research emerge from this work:

- **Dataset Expansion:** Collect additional handwritten samples for under-represented classes to improve recognition of rare upazilas and further balance the class distribution across all 496 administrative pairs.
- **Improved Segmentation:** Investigate deep learning-based segmentation methods to reduce the approximately 10% of cases requiring manual intervention, potentially through end-to-end trainable detection models.
- **Model Optimization:** Explore model compression and knowledge distillation techniques to reduce computational requirements and enable deployment on resource-constrained edge devices.
- **Full Address Recognition:** Extend the system to recognize complete handwritten addresses (house number, road name, postcode, etc.) beyond just upazila–district pairs.
- **Language Model Integration:** Incorporate Bangla language models for post-processing correction based on valid district-upazila combinations to further improve recognition accuracy in practical applications.

REFERENCES

[1] "Bengali language," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Bengali_language.

[2] A. S. M. Rabby, S. Hossain, S. Islam, K. Abujar, and S. A. Hossain, "Bormonet: Bangla handwritten character recognition using CNN," *Procedia Computer Science*, vol. 143, pp. 528–535, 2018.

[3] T. Tazin, S. S. Sajib, and M. R. Karim, "A Scalable OCR Pipeline for Bangla Handwritten Documents," *Discover Artificial Intelligence*, vol. 5, art. no. 251, 2025, doi:10.1007/s44163-025-00251-7.

[4] A. Dosovitskiy et al., "An Image Is Worth 16×16 Words: Transformers for Image Recognition at Scale," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2021.

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 11976–11986.

[6] S. M. Mahbubur Rahman et al., "Bangla Handwritten Character Recognition Using Convolutional Neural Network," in *Proc. Int. Conf. Artificial Intelligence and Applications (ICAIA)*, 2022.

[7] M. A. Hamid et al., "Bangla Handwritten Character Recognition Using a Modified ResNet-34 Architecture," *Int. J. Adv. Comput. Sci. Appl.*, vol. 14, no. 2, pp. 123–130, 2023.

[8] Md. H. Zisad et al., "BNHW-UDPNR: A Dataset of Handwritten Bengali District Names for Text Recognition," *Data in Brief*, vol. 48, art. no. 109123, 2025.

[9] M. Kowsher et al., "End-to-End Automated Bangla Signboard Detection and Categorization System," *Results in Engineering*, vol. 26, art. no. 102319, 2025.

[10] M. R. Islam et al., "RATNet: A Heterogeneous Deep Learning Model for Bengali Character Recognition," *Multimedia Tools Appl.*, vol. 81, pp. 19661–19694, 2022, doi:10.1007/s11042-022-12070-4.

[11] A. Mollah, A. Basu, and S. Bhowmik, "Handwritten Bangla City Name Recognition Using Histogram of Oriented Gradients and Support Vector Machine," in *Proc. Int. Conf. Computer, Communication and Electrical Technology (ICCET)*, 2021.

[12] M. T. Habib et al., "A Comparative Study of Machine Learning Algorithms for Bangla Handwritten Character Recognition," *J. King Saud Univ. Comput. Inf. Sci.*, vol. 34, no. 6, pp. 3456–3467, 2022.

[13] U. Pal and B. B. Chaudhuri, "Indian Script Character Recognition: A Survey," *Pattern Recognition*, vol. 37, no. 9, pp. 1887–1899, 2004.

[14] M. Z. Alom and V. K. Asari, "Handwritten Bangla Character Recognition Using the State-of-the-Art Deep Convolutional Neural Networks," *Comput. Intell. Neurosci.*, vol. 2018, art. no. 6747098, 2018.

[15] S. Bag and G. Harit, "A Survey on Optical Character Recognition for Bangla and Devanagari Scripts," *Sādhanā*, vol. 38, no. 1, pp. 133–168, 2013.

[16] R. Sarkar et al., "A Two-Stage Approach for Segmentation of Handwritten Bangla Word Images," *Int. J. Comput. Inf. Syst. Ind. Manag. Appl.*, vol. 4, pp. 403–414, 2012.

[17] N. Das et al., "A Statistical-Topological Feature Combination for Recognition of Handwritten Bangla Compound Characters," in *Proc. Int. Conf. Adv. Pattern Recognit. (ICAPR)*, 2009, pp. 150–153.

[18] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[19] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2015.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[21] G. Huang et al., “Densely Connected Convolutional Networks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 4700–4708.

[22] A. Vaswani et al., “Attention Is All You Need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[23] J. Yosinski et al., “How Transferable Are Features in Deep Neural Networks?” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

[24] S. J. Pan and Q. Yang, “A Survey on Transfer Learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[25] T. Kang et al., “Pay Attention to What You Read: Non-Recurrent Handwritten Text-Line Recognition,” *Pattern Recognition*, vol. 129, art. no. 108766, 2022.

[26] T. G. Dietterich, “Ensemble Methods in Machine Learning,” in *Proc. Int. Workshop Multiple Classifier Systems (MCS)*, 2000, pp. 1–15.

[27] L. K. Hansen and P. Salamon, “Neural Network Ensembles,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 12, no. 10, pp. 993–1001, Oct. 1990.

[28] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*. Boca Raton, FL, USA: CRC Press, 2012.

[29] R. E. Schapire, “The Boosting Approach to Machine Learning: An Overview,” in *Nonlinear Estimation and Classification*. New York, NY, USA: Springer, 2003, pp. 149–171.

[30] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Ann. Stat.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[31] Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.

[32] D. H. Wolpert, “Stacked Generalization,” *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992.

[33] L. Breiman, “Bagging Predictors,” *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, 1996.

[34] L. Breiman, “Random Forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[35] J. Kittler et al., “On Combining Classifiers,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 3, pp. 226–239, 1998.

[36] G. Fumera and F. Roli, “A Theoretical and Experimental Analysis of Linear Combiners for Multiple Classifier Systems,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 6, pp. 942–956, 2005.

[37] D. Ruta and B. Gabrys, “An Overview of Classifier Fusion Methods,” *Comput. Inf. Syst.*, vol. 7, no. 1, pp. 1–10, 2000.

[38] H. Zhang et al., “Mixup: Beyond Empirical Risk Minimization,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2018.

[39] C. Szegedy et al., “Rethinking the Inception Architecture for Computer Vision,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2818–2826.

[40] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2019.

[41] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.

[42] N. Srivastava et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[43] G. Huang et al., “Deep Networks with Stochastic Depth,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2016, pp. 646–661.

[44] R. Wightman, “PyTorch Image Models,” GitHub repository, 2019. [Online]. Available: https://github.com/rwightman/pytorch-image-models.

[45] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in *Proc. Conf. Empirical Methods Natural Language Process. (EMNLP): System Demonstrations*, 2020, pp. 38–45.

[46] GeeksforGeeks, "ConvNeXt Architecture," Available: https://www.geeksforgeeks.org/computer-vision/convnext/.

[47] O. Katar and O. Yildirim, "An explainable vision transformer model based white blood cells classification and localization," *Diagnostics*, vol.13, no. 14, art. no. 2459, Jul. 2023. doi: 10.3390/diagnostics13142459.